

UNIVERSIDADE FEDERAL DO TOCANTINS
CÂMPUS UNIVERSITÁRIO DE PALMAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

GERAÇÃO DE MÚSICAS PARA VIDEOGAME COM O USO DE REDES NEURAIS RECORRENTES LSTM

JOÃO VITOR RIBEIRO DA SILVA

PALMAS (TO)
2021

JOÃO VITOR RIBEIRO DA SILVA

GERAÇÃO DE MÚSICAS PARA VIDEOGAME COM O USO DE REDES NEURAIS RECORRENTES LSTM

Trabalho de Conclusão de Curso II apresentado à Universidade Federal do Tocantins para obtenção do título de Bacharel em Ciência da Computação, sob a orientação do(a) Prof.(a) Dra. Anna Paula de Sousa Parente Rodrigues.

Orientador: Dra. Anna Paula de Sousa Parente Rodrigues

PALMAS (TO)

2021

JOÃO VITOR RIBEIRO DA SILVA

GERAÇÃO DE MÚSICAS PARA VIDEOGAME COM O USO DE REDES NEURAIS RECORRENTES LSTM

Trabalho de Conclusão de Curso II apresentado à UFT – Universidade Federal do Tocantins – Câmpus Universitário de Palmas, Curso de Ciência da Computação foi avaliado para a obtenção do título de Bacharel e aprovada em sua forma final pelo Orientador e pela Banca Examinadora.

Data de aprovação: 16 / 12 / 2021

Banca Examinadora:

Profa. Dra. Anna Paula de Sousa Parente Rodrigues

Prof. Dr. Marcelo Lisboa Rocha

Prof. Dr. Warley Gramacho da Silva

A alguém cujo valor é digno

desta dedicatória.

AGRADECIMENTOS

Gostaria de agradecer a todos.

RESUMO

As redes neurais LSTM (acrônimo para *Long Short-Term Memory*) são um tipo de rede neural recorrente, ou seja, um tipo de rede neural onde as informações que são recebidas como entrada persistem por um determinado tempo afetando a *hidden layer* e a saída do modelo através de um ciclo de realimentação, porém são superiores às RNN padrão, possuindo mecanismos de retenção de informações de longo prazo, sendo mais complexas que as RNN padrão, porém muito eficientes para captar informações chaves dentro de informações seriais. São muito utilizadas em diversas aplicações, incluindo as de análise de sentimentos, reconhecimento de fala e predições temporais.

O objetivo do presente trabalho é dar um embasamento teórico sólido e esboçar um plano metodológico para a realização da segunda etapa do trabalho e também dar uma estimativa sobre os resultados esperados e um cronograma descrevendo os próximos passos para a sua concretização, que inclui o desenvolvimento e avaliação de um protótipo LSTM.

Palavra-chave: Aprendizado Profundo. Música para Videogames. Redes Neurais Recorrentes. Redes LSTM. Geração de Música com Aprendizado de Máquina.

ABSTRACT

The LSTM (acronym for Long Short-Term Memory) neural networks are a type of recurrent neural network, that is, a type of neural network that maintains the input information and its effects on the the hidden layer and on the output of the neural network for a certain period of time through feedback loops. The LSTM is superior to the vanilla recurrent neural networks because the LSTM model has long-term information retention mechanisms, therefore are more complex than the vanilla RNN and are much more effective for the captation of key components within serial information. The LSTM are largely used in many applications, including those regarding sentiment analysis, speech recognition and time series predictions.

The objective of the present work is to give a solid theoretical basis and trace a methodological plan for the achievement of the second stage of the work and also return estimates about the expected results and a schedule depicting the next steps for its completion, which includes the creation and evaluation of a LSTM prototype and its products.

Keywords: Deep Learning. Videogame Music. Recurrent Neural Networks. LSTM Networks. Music Generation with Machine Learning.

LISTA DE FIGURAS

Figura 1 – Duas pautas musicais, uma com a clave de Sol e a outra com a clave de Fá.[1] 20
Figura 2 – Hierarquia das Figuras Rítmicas.[2] 22
Figura 3 – Escala maior de C representada em piano.[3] 23
Figura 4 – Círculo das quintas, escala em questão - C maior.[4] 24
Figura 5 – Neurônio biológico e neurônio artificial. [5] 26
Figura 6 – Representação gráfica da convergência do perceptron em diferentes portas lógicas.[6] 28
Figura 7 – Representação gráfica da função de ativação Sigmoide, que independentemente do valor de entrada, retorna um número entre 0 e 1. . . 30
Figura 8 – Exemplo do ajuste de pesos na direção inversa ao gradiente. 32
Figura 9 – Elman x Jordan. [7] 33
Figura 10 – Representação gráfica de uma rede neural recorrente simples. 34
Figura 11 – Célula LSTM e seus portões.[8] 35
Figura 12 – Representação gráfica de uma mensagem MIDI.[9] 39

Figura 13 – Representação gráfica do pré-processamento de dados. 42
Figura 14 – Treinamento da Rede LSTM. 46
Figura 15 – Inferência Musical com a Rede Neural Treinada. 48

Figura 16 – Capa do Jogo “Dragon View”.[10] 52
Figura 17 – Eventos nas *seeds* das quatro músicas - “Dragon View”. 52
Figura 18 – Quantidades de eventos por tipo nas quatro músicas - “Dragon View”. 53
Figura 19 – Quantidade de eventos por tom nas músicas - “Dragon View”. 53
Figura 20 – Sequência de eventos nas quatro músicas - “Dragon View”. 54
Figura 21 – Capa do Jogo “Final Fantasy III”.[11] 55
Figura 22 – Eventos nas *seeds* das quatro músicas - “Final Fantasy III”. 56
Figura 23 – Quantidades de eventos por tipo nas quatro músicas - “Final Fantasy III”. 56

Figura 24 – Quantidade de eventos por tom nas músicas - “Final Fantasy III”. . . 57
Figura 25 – Sequência de eventos nas quatro músicas - “Final Fantasy III”. 57
Figura 26 – Eventos nas *seeds* das quatro músicas - Dados de Tipo 2. 60
Figura 27 – Quantidades de eventos por tipo nas quatro músicas - Dados de Tipo 2. 61
Figura 28 – Quantidade de eventos por tom nas músicas - Dados de Tipo 2. . . . 61
Figura 29 – Sequência de eventos nas quatro músicas - Dados de Tipo 2. 62

LISTA DE TABELAS

Tabela 1 – Tabela de oitavas. 19
Tabela 2 – Tabela da porta lógica XOR. 27

SUMÁRIO

1 INTRODUÇÃO 13

1.1 Justificativa 14

1.2 Objetivos 15

1.2.1 Objetivo Geral 15

1.2.2 Objetivos Específicos 15

1.3 Revisão Bibliográfica 16

1.4 Organização do Trabalho 16

2 REFERENCIAL TEÓRICO 18

2.1 Teoria musical 18

2.1.1 Tons, Notas e Oitavas 18

2.1.2 Pauta e Notação Musical 19

2.1.3 Escalas e Acordes 23

2.2 Redes Neurais Artificiais 24

2.2.1 Neurônio Artificial vs. Neurônio Humano 24

2.2.2 Perceptron 26

2.2.3 Multilayer Perceptron 29

2.2.4 Redes Neurais Recorrentes 31

2.2.5 Redes Neurais Recorrentes Long Short-Term Memory (LSTM) 35

2.3 Formato MIDI 36

2.3.1 Canais MIDI 37

2.3.2 Modos e Estados MIDI 37

2.3.3 Mensagens (Comandos) MIDI 37

3 METODOLOGIA 40

3.1 Ferramentas Utilizadas 40

3.1.1 Hardware 40
3.1.2 Software 40
3.1.2.1 Linguagem de Programação Python 40
3.1.2.2 MuseScore 41
3.2 Coleta de Dados 41
3.3 Pré-Processamento dos Arquivos MIDI 42
3.4 Modelagem da Rede Neural 45
3.4.1 Treinamento e Validação da Rede Neural 45
3.4.2 Inferência de Dados com a LSTM 47
3.5 Pós-Processamento 49
4 RESULTADOS 51
4.1 Avaliando a Geração Musical com os Dados do Tipo 1 51
4.1.1 Resultados Obtidos com Músicas do Jogo "Dragon View" - Dados do Tipo 1 51
4.1.2 Resultados Obtidos com Músicas do Jogo "Final Fantasy III" - Dados do Tipo 1 55
4.1.3 Análise dos Resultados - Dados do Tipo 1 57
4.2 Avaliando a Geração Musical com os Dados do Tipo 2 60
5 CONCLUSÃO E TRABALHOS FUTUROS 64
5.1 Trabalhos Futuros 64
REFERÊNCIAS 65

1 INTRODUÇÃO

Graças às mudanças tecnológicas, tanto de hardware quanto de software, os sons produzidos pelos videogame evoluíram muito conforme o tempo, entre o simples "beep"dos jogos da época de *Pong*, que eram feitos com circuitos digitais, e as as sinfonias que existem nos jogos mais modernos como *Dark Souls*, há uma grande variedade de metodologias e processos, porém, de acordo com (COLLINS, 2008), não há um ponto exato onde começa a se haver músicas em videogames, pois esta definição vai depender do que pode ser considerado música ou não, porém o autor também cita os jogos *Space Invaders* e *Asteroids* como expoentes importantes devido, respectivamente, sua música contínua e melodias de duas notas.

De acordo com (COLLINS, 2005), a história da música dos videogames se divide em 3 etapas principais: a era dos sons em 8-bits, que são os "beeps"e "bops"dos primeiros videogames, a segunda que se trata da expansão do 8-bit para o 16-bit e da criação do formato MIDI - Musical Digital Interface, e a terceira que é a etapa da inclusão dos cds de áudio Redbook, que permeia até a atualidade.

O formato MIDI é um formato de arquivo de som que define padrões musicais e que mudou a maneira com a qual os instrumentos musicais são reproduzidos e processados por computadores, permitindo que diferentes instrumentos se comunicassem através do padrão MIDI, onde, por exemplo, um cabo MIDI pode transmitir até dezesseis canais de informações, cada um com diferentes instrumentos, capacidade que permite a melhor gravação e pós-produção musical (mixagem e masterização). O formato musical MIDI foi criado em 1983 a partir de preocupações vindas de setores da indústria musical que se preocupavam com a falta de compatibilidade entre instrumentos de diversas marcas, que anteriormente eram incompatíveis, ou seja, um músico não poderia utilizar módulos sintetizadores em instrumentos que não fossem da mesma marca, o que fazia com que as vendas não atingissem seu potencial total (ASSOCIATION, 2014-2021).

Para (MAZZOLA et al., 2018), o aspecto mais genial do código MIDI é não se preocupar em representações muito abstratas da música, mas sim em codificar a ação gestual de se tocar música, visto que a indústria musical não estava interessada em símbolos complexos, mas sim em uma maneira de fazer com que os instrumentos conversassem a mesma linguagem digital. Segundo (UITDENBOGERD; ZOBEL, 1998), conseguir extrair informações a partir de bases de dados musicais e utilizá-las para realizar operações e avaliações é útil para diversas aplicações, desde verificações de violações de direitos autorais até a realização de estudos de musicologia.

O primeiro computador a ser capaz de reproduzir músicas foi o CSIR Mark 1, planejado e construído por Trevor Pearcey e Maston Beard, na Austrália durante a década

de 50 e que foi programado para tocar músicas pelo matemático Geoff Hill, porém a exploração da possibilidade de reprodução musical em computadores ocorria simultaneamente na Inglaterra e nos Estados Unidos, em projetos da Victoria University of Manchester e da Bell Labs, respectivamente (DOORNBUSCH, 2017). As máquinas dessa época funcionavam em baixo nível e tinham que ter as notas musicais programadas diretamente nelas para que a música fosse tocada, porém deixaram o legado da música reproduzida por computadores que também pode ser interpretado como música em formato digital, o que abre um grande leque de possibilidades computacionais, como a criação e a edição de músicas diretamente pelo computador.

Dessa forma, o presente trabalho de conclusão de curso tem por objetivo a geração automática de músicas de vídeo game através de técnicas de aprendizado de máquina, mais especificamente, por meio de redes neurais artificiais do tipo recorrente. Tais redes são uma classe de rede neural amplamente utilizada no reconhecimento de fala e processamento de linguagem natural e, de acordo com (CARLING, 1992), as redes neurais recorrentes possuem características importantes, como poder reutilizar cálculos previamente feitos na computação atual, permitir que a rede retorne saídas complexas a partir de entradas simples.

1.1 Justificativa

A partir de todo o processo de evolução musical e com o advento dos algoritmos de inteligência artificial obtivemos a possibilidade de geração musical automatizada. A ideia da geração musical automatizada não é nova, existem praticamente desde o começo da descoberta da possibilidade de reprodução musical em computadores, como é o exemplo da *Illiac Suite*, um modelo estocástico de composição musical para quatro instrumentos de corda desenvolvido em 1957 por Lejaren Hiller e Leonard Issacson. No entanto, os avanços computacionais indicam um novo horizonte na era da música digital, que assim como na era atual, utiliza da tecnologia disponível para criar e modificar músicas de maneira não-automática, utilizará da abundância de dados e das capacidades computacionais disponíveis para criar ou auxiliar a criação de melodias e composições.

De acordo com (PAPADOPOULOS; WIGGINS, 1999), os tipos de sistemas utilizados na música podem ser dos mais diferentes possíveis, como os matemáticos (a exemplo do *Illiac Suite*), os modelos baseados em conhecimento, em gramáticas, métodos evolutivos, sistemas que aprendem e os métodos híbridos. Um dos expoentes mais famosos dos experimentos de geração musical através de computadores foi o EMI - Experiments in Musical Intelligence, que foi criado antes da era do Deep Learning e que se enquadra na classificação de (PAPADOPOULOS; WIGGINS, 1999) como um modelo baseado em gramática e é descrito por (COPE; SILVA, 2013) como um programa que, apesar de não inteligente, conseguiu produzir música convincente onde cada composição é resultado da

seleção de um conjunto finito de limitações.

Assim, na execução do presente trabalho de conclusão de curso serão utilizadas as redes neurais LSTM - Long Short Term Memory, que são um tipo de RNN, de forma a gerar automaticamente músicas de vídeo game. As redes LSTM foram escolhidas uma vez que possuem mecanismos extras para saber qual é o momento certo para se reter uma informação e quando se desfazer da mesma, capacidades essenciais para a predição musical, onde se lembrar de informações como notas e acordes anteriores e saber quando a emoção ou as notas da música devem mudar é essencial.

Outro fato a ser levado em consideração para o desenvolvimento do trabalho se encontra na praticidade da geração musical de jogos automatizada, que para equipes de criação de jogos independentes pode ser muito útil, pois estes times geralmente são reduzidos e focam na parte do software e da arte visual do jogo, deixando as trilhas sonoras para segundo plano, dessa forma, o desenvolvimento da aplicação se justifica no auxílio à produção musical para estas equipes, seja no desenvolvimento completo de todas as músicas que incluem a trilha sonora ou na criação dos arquivos MIDI que servirão de base para o seu desenvolvimento, permitindo o uso da criatividade pela parte dos desenvolvedores.

1.2 Objetivos

De maneira a manter a organização deste trabalho de conclusão de curso, os objetivos foram divididos em geral e específicos.

1.2.1 Objetivo Geral

Geração automática de músicas para videogame utilizando redes neurais do tipo LSTM.

1.2.2 Objetivos Específicos

De forma a atingir os objetivos específicos, as seguintes etapas são propostas:

1. Coleta de músicas de videogame no formato MIDI ou conversão de músicas de videogame em formatos diferentes para o formato MIDI, com o intuito de obter uma base de dados sólida;

2. Aplicar técnicas de pré-processamento para os dados musicais anteriormente coletados para a obtenção de dados que podem ser utilizados no treinamento e validação do formato MIDI;

3. Estudo e implementação de redes neurais LSTM com os dados previamente processados para que esta possa ser treinada e validada;

4. Geração de músicas a partir da rede neural treinada e a avaliação da qualidade das músicas obtidas usando métricas da teoria musical.

1.3 Revisão Bibliográfica

Em (MANGAL; MODAK; JOSHI, 2019) é demonstrada uma maneira de criação de músicas a partir de arquivos MIDI com o uso de redes neurais *Long Short-Term Memory*, usando técnicas de pré-processamento de arquivos MIDI que envolviam limpeza e normalização de notas e matrização de notas musicais, transformando-as em tensores 3D e fazendo possível com que o modelo LSTM possa gerar predições sobre as próximas notas, e em (ZHAO et al., 2019), que se refere a captação de emoção em músicas com o uso da tecnologia LSTM, também são demonstradas técnicas interessantes de pré-processamento de arquivos MIDI relevantes ao presente trabalho, como representação a matricial de notas musicais.

Outro método utilizado para a geração de músicas com o uso das LSTM pode ser encontrado em (CHOI; FAZEKAS; SANDLER, 2016), que discute o uso de métodos textuais para a interpretação de músicas por uma LSTM, utilizando a progressão de acordes e da bateria em dois modelos, o primeiro analisando os eventos musicais como palavras e o segundo analisando-os como caracteres.

Existem também a possibilidade de uso um tipo ainda mais específico de rede neural, que mistura as LSTM com as máquinas de Boltzmann restritas, chamadas de LSTM-RTRBM, como apresentado em (LYU; WU; ZHU, 2015), que é utilizada para aprimorar a capacidade polifônica das geração musical.

1.4 Organização do Trabalho

Este trabalho será organizado em capítulos que abordam os assuntos, pré-processamento de dados musicais, geração automática de músicas em formato MIDI e redes neurais artificiais do tipo recorrente - LSTM. De forma mais específica, o trabalho é dividido em X capítulos, com a seguinte formatação:

1. Capítulo 1 - Introdução: diz respeito à introdução do presente trabalho, nele são abordados o contexto histórico de computação musical, do desenvolvimento de músicas para videogames, do uso de aplicações de inteligência artificial para a geração de músicas bem como sobre a justificativa para o desenvolvimento e informações gerais (introdutórias) sobre a razão da escolha das LSTM.

2. Capítulo 2 - Referencial Teórico: diz respeito ao referencial teórico por trás do desenvolvimento do presente trabalho, incluindo conceitos de teoria musical, tanto para a explicação de conceitos utilizados agora quanto para a clarificação sobre métricas utilizadas no segundo trabalho. Também fala sobre a teoria envolvendo as

redes neurais, que vai desde explicações sobre o neurônio artificial e se afunila na teoria por trás das redes neurais recorrentes LSTM.

3. Capítulo 3 - Metodologia: que visa explicar a maneira com a qual o que foi exposto no segundo capítulo será utilizado na segunda etapa do trabalho, demonstrando as etapas de pré-processamento, treinamento e validação da rede neural, pós-processamento e avaliação do conteúdo musical obtido.

4. Capítulo 4 - Considerações Finais: busca trazer os objetivos para o próximo trabalho, ligando o que foi exposto nesta etapa do trabalho com o que será desenvolvido na próxima, apresentando também um cronograma de atividades.

2 REFERENCIAL TEÓRICO

Este capítulo tem o intuito de apresentar os conceitos fundamentais utilizados na criação deste trabalho, incluindo o seguinte conteúdo: teoria musical, que engloba os aspectos específicos que serão utilizados no presente trabalho e na segunda etapa; redes neurais artificiais, dado um panorama geral sobre as tecnologias que precedem e culminam nas redes LSTM; e uma explicação sobre o formato MIDI e sua arquitetura, algo necessário para se compreender como o pré-processamento e a manipulação sobre estes dados ocorre.

2.1 Teoria musical

A definição de música em si é algo complicado, pois esta pode ser descrita de diversas maneiras que podem variar de acordo com pessoas e sociedades e o modo como estas mudam com o passar do tempo, porém uma definição de música que pode ser abrangente o suficiente para satisfazer os critérios mais amplos é que "a música é o som organizado". A teoria musical se trata então do estudo sobre esta forma de arte racionalizando-a e a transformando em um assunto teórico, criando padrões como notas, acordes e harmonias que podem ser generalizados e aplicados aos mais distintos timbres e, de acordo com (SCHMIDT-JONES, 2013), pode ser comparada à gramática, onde as linguagens são inventadas pelas pessoas que a falam e com o passar do tempo as sociedades acabam por criar esquematizações e regras para defini-las e facilitar sua compreensão e seu uso, dessa forma, um compositor musical pode ser comparado a um escritor, onde ambos dominam o seu campo de estudo e podem então criar arte a partir das regras gramaticais ou musicais que os antecedem.

Esta seção abordará o tema de teoria musical e como ela se aplica ao presente trabalho, pois noções básicas de teoria musical se darão necessárias para entendê-lo, em especial dos conceitos de notação musical, notas, acordes e escalas.

2.1.1 Tons, Notas e Oitavas

As notas musicais são doze: Dó, (Dó sustenido ou Ré bemol), Ré, (Ré sustenido ou Mi bemol), Mi, Fá, (Fá sustenido ou Sol bemol), Sol, (Sol sustenido ou Lá bemol), Lá, (Lá sustenido ou Si bemol) e Si, porém no presente trabalho estas serão descritas na notação científica de altura, onde as notas são representadas por letras, sendo as notas naturais: C = Dó, D = Ré, E = Mi, F = Fá, G = Sol, A = Lá e B = Si. As notas acidentais sustenidas: C# = Dó Sustenido, D# = Ré Sustenido, F# = Fá Sustenido, G# = Sol Sustenido e A# = Lá Sustenido. As notas acidentais bemóis: D_b = Ré Bemol, E_b = Mi Bemol, G_b = Sol Bemol, A_b = Lá Bemol e B_b = Si Bemol.

As notas acidentais tem esse nome pois estas ocorrem para aumentar (sustenido) ou diminuir (bemol) uma determinada nota musical em um semitom, e é daí que surgem

os 5 acidentais. Os acidentais podem ser intercambiáveis, sendo que C# = D_b, D# = E_b, F# = G_b, G# = A_b e A# = B_b, e estes são utilizados dependendo da nota anterior, quando o tom da nota anterior é aumentado, o valor é sustenido e quando o tom da nota anterior é diminuído, o valor é bemol, da seguinte forma: C-C#-D-D#-E-F-F#-G-G#-A-A#-B para as notas e seus sustenidos, com as notas indo na direção dos agudos ($\rightarrow$), e C-D_b-D-E_b-E-F-G_b-G-A_b-A-B_b-B para as notas e seus bemóis, com as notas indo na direção dos graves ($\leftarrow$). Cada nota musical representa uma frequência difrente (em hertz), sendo que o espaço entre uma nota e outra é chamado de semitom e o espaço entre duas notas é chamado de tom, a variação de frequência entre o valor de uma nota e outra nota um semitom acima pode ser definida como $f = f_{na} \times \sqrt[12]{2}$ onde f é a frequência da próxima nota e f_{na} é a frequencia da nota atual.

As oitavas, na notação científica de altura, são definidas como o conjunto de doze notas que pertencem a uma faixa específica de frequências, começando com a nota C e terminando na nota B, caso a frequência desta última nota B seja aumentada, como demonstrado no parágrafo anterior, a próxima nota será C, porém uma oitava acima da oitava de B.

Tabela 1 – Tabela de oitavas.

Nota/Oitava	1	2	3	4	5	6	7
C	32.70320	65.40639	130.8128	261.6256	523.2511	1046.502	2093.005
C#/D_b	34.64783	69.29566	138.5913	277.1826	554.3653	1108.731	2217.461
D	36.70810	73.41619	146.8324	293.6648	587.3295	1174.659	2349.318
D#/E_b	38.89087	77.78175	155.5635	311.1270	622.2540	1244.508	2489.016
E	41.20344	82.40689	164.8138	329.6276	659.2551	1318.510	2637.020
F	43.65353	87.30706	174.6141	349.2282	698.4565	1396.913	2793.826
G	46.24930	92.49861	184.9972	369.9944	739.9888	1479.978	2959.955
G#/A_b	48.99943	97.99886	195.9977	391.9954	783.9909	1567.982	3135.963
G#/A_b	51.91309	103.8262	207.6523	415.3047	830.6094	1661.219	3322.438
A	55.00000	110.0000	220.0000	440.0000	880.0000	1760.000	3520.000
A#/B_b	58.27047	116.5409	233.0819	466.1638	932.3275	1864.655	3729.310
B	61.73541	123.4708	246.9417	493.8833	987.7666	1975.533	3951.066

A Tabela 1 acima mostra algumas das oitavas possíveis e atribui a cada nota um valor de frequência em hertz e a qual oitava a nota pertence, sendo que a nota considerada central é a C_4, que emite uma frequência de 261,6256 hertz, como dito em (SCHMIDT-JONES, 2013).

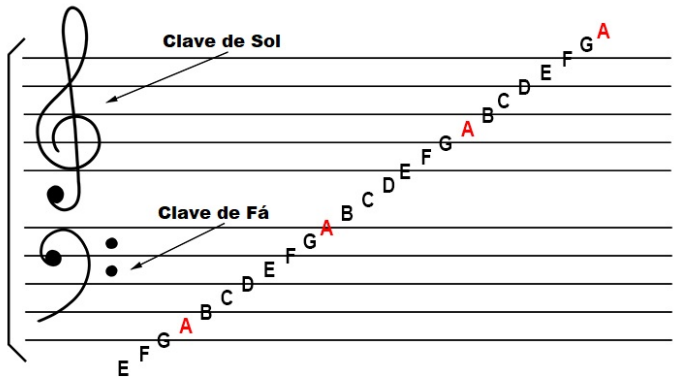
2.1.2 Pauta e Notação Musical

A música é comumente descrita através da pauta, ou petangrama, que nada mais é que o nome do conjunto de cinco linhas horizontais paralelas utilizado para se ajudar a descrever uma melodia, a pauta é lida da esquerda para a direita e contém barras verticais

que podem ser simples ou duplas. Estas notações serão utilizadas no segundo trabalho para a representação das melodias: barras simples, que indicam os compassos da música, barras simples duplas que indicam o começo de outra seção da música, barras finais que final indicam o final da música.

Outra parte importante para a notação musical é a clave, que pode variar, porém para a avaliação dos resultados obtidos no segundo trabalho, serão utilizadas a clave de Sol: 𝄞 e a clave de Fá: 𝄢 , que servem de maneira prática como uma âncoras para as outras notas musicais na pauta, sendo que a clave de Sol aparece na segunda linha da pauta, indicando que aquela representa a nota Sol (G), portanto, o espaço entre esta linha e a próxima representa a próxima nota em relação a nota Sol (Lá/A) e o espaço entre esta linha e a linha anterior representa a nota anterior (Fá/F) e assim sucessivamente, e a clave de Fá (F) aparece na quarta linha, indicando a nota de Fá, ambas organizam as notas na pauta da seguinte maneira:

Figura 1 – Duas pautas musicais, uma com a clave de Sol e a outra com a clave de Fá.[1]



Caso haja a necessicade de adição de notas fora do alcance da oitava onde a clave se encontra, linhas e espaços adicionais podem ser utilizados, tanto para cima quanto para baixo a pauta.

O uso das duas claves é importante pois a clave de Sol representa a nota G acima da nota C central e a clave de Fá representa a nota F abaixo da nota C central, dessa forma, a clave de Fá geralmente é utilizada para a escrita e leitura de instrumentos mais graves, tanto que seu nome em inglês é *bass clef* (clave de baixo).

Um item que geralmente segue as claves são as armaduras de clave, que são chamadas de *key signature* (assinatura de chave) em inglês e têm a função de indicar a qual escala a melodia pertence. As armaduras de clave servem para colocar as notas acidentais nos locais corretos, colocando os símbolos # e $_b$ nas linhas ou espaços, mudando o valor

[1]Mammoth Memory. Disponível em: <https://mammothmemory.net/music/sheet-music/reading-music/treble-clef-and-bass-clef.html>, Acesso em nov. 2021

destas, por exemplo, caso a clave em questão seja a de Sol e caso haja um símbolo "#" na terceira linha, que representa a nota B, todas as notas atribuidas e esta linha serão B#. Por exemplo, a armadura de clave para a escala G maior (notas G, A, B, C, D, E, F#), com a clave de Sol:



Após as armaduras de clave, o próximo elemento que aparece na pauta é a assinatura de tempo, que aparece somente uma vez na pauta, diferentemente da clave e da armadura de clave, que aparecem no começo de todas as linhas de pauta que compõem a música. A assinatura de tempo é composta por dois números que ficam um sobre o outro e são lidos como "assinatura de tempo (*número superior*) por (*número inferior*)", então a assinatura de tempo três por quatro se mostra na pauta da seguinte forma:



O numero superior indicado na assinatura de tempo é o número de batidas em uma medida (ou barra) e o número inferior indica o valor correspondente ao valor de uma medida. As assinaturas de tempo se parecem com frações e são, de fato, maneiras de se preencher cada medida com o número superior, usando o exemplo da assinatura de tempo três quartos, poderão usadas três notas que correspondem a 14 para se preencher o espaço da medida, bem como uma nota que corresponde a 12 e uma nota que corresponde a 14.

Por fim, na pauta, as figuras que servem para apontar qual nota musical deve ser tocada e por quanto tempo esta deve ser tocada (ou por quanto tempo não se deve tocar nota musical alguma, no caso de uma pausa) é a figura rítmica, que são símbolos que indicam valores de tempo em uma música. As figuras que poderão ser utilizadas na segunda etapa do trabalho são:

- Semibreve (𝅝) que é chamada de *whole note* (nota inteira) em inglês e é a figura considerada base para a duração de tempo das outras, que comumente é assumida com o valor 4 pois o valor da assinatura de tempo geralmente é (*número X*) por quatro, sendo que a semibreve corresponde pela duração inteira do número inferior da assinatura de tempo, ocupando assim quatro batidas de tempo. Sua equivalente em pausa é a 𝄻;

- Mínima (𝅗𝅥), que corresponde a metade de uma semibreve e no caso desta valer 4 (caso considerado padrão), a mínima vale 2. Sua equivalente em pausa é a 𝄼;

- Semínima (♩), que corresponde a metade de uma semibreve e no caso desta valer 2, a semínima vale 1. Sua equivalente em pausa é a 𝄽;

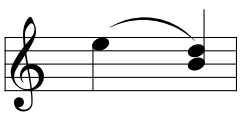
- Colcheia (♪), que corresponde a metade de uma semínima e no caso desta valer 1, a colcheia vale 12;
- Semicolcheia (♬), que corresponde a metade de uma colcheia e no caso desta valer 12, a semicolcheia vale 14. Sua equivalente em pausa é a 𝄾;
- Fusa (𝅘𝅥𝅯), que corresponde a metade de uma semicolcheia e no caso desta valer 14, a fusa vale 18. Sua equivalente em pausa é a 𝄿;
- Semifusa (𝅘𝅥𝅰), que corresponde a metade de uma fusa e no caso desta valer 18, a semifusa vale 116. Sua equivalente em pausa é a 𝅀;

E podem ser descritas visualmente de acordo com o gráfico:

Figura 2 – Hierarquia das Figuras Rítmicas.[2]

Figuras Musicais				
Desenho	Nome	Valor Relativo	Pausas	Representação do Valor Relativo
	Semibreve	1		
	Mínima	2		
	Semínima	4		
	Colcheia	8		
	Semicolcheia	16		
	Fusa	32		
	Semifusa	64		

Outras notações que serão utilizadas na segunda etapa do trabalho serão as notações de Naturais (♮), que cancelam um sustenido ou bemol em uma determinada linha ou espaço onde estes deveriam ser tocados, e vem antes de uma figura musical da seguinte forma (♮♩). Outra notação que também será utilizada é a notação do ponto (𝅭), que pode acompanhar as figuras rítmicas e adiciona metade da duração da figura musical sobre si mesma, ou seja: $\dot{\text{♩}} = \text{♩} + \text{♪}$. Outro tipo de notação que pode ser utilizada na segunda etapa do trabalho é a ligadura, que, como seu próprio nome diz, liga uma figura musical a outra pelo tempo definido, caso estas estejam na mesma altura (tocando a mesma nota), o som apenas se prolonga, caso contrário ela transiciona de uma nota para a outra, a ligadura é descrita como um arco entra duas notas da seguinte forma:

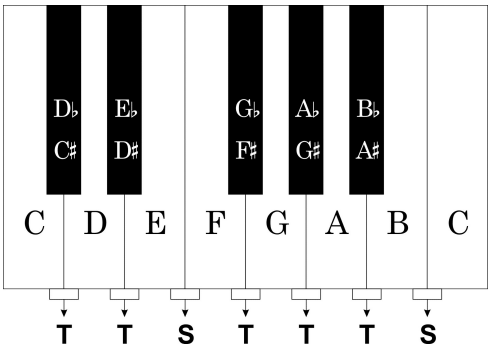


[2] PraMusic. Disponível em: <https://pramusic.wordpress.com/a-musica/figuras-musicais/figuras-ritmicas/>, Acesso em dez. 2021

2.1.3 Escalas e Acordes

As escalas podem ser definidas de maneira simples como uma sequência de notas musicais ordenadas e que seguem uma determinada sequência de intervalos entre suas notas, explicando de maneira prática, pode-se usar como exemplo a escala maior, que mantém a transição entre suas notas como: Tom-Tom-Semitom-Tom-Tom-Tom-Semitom (Um tom equivale a dois semitons), logo, a escala maior de C poderia ser representada como: C-C#/Db-D-D#/Eb-E-F-F#/Gb-G-G#/Ab-A-A#/Bb-B-C.

Figura 3 – Escala maior de C representada em piano.[3]



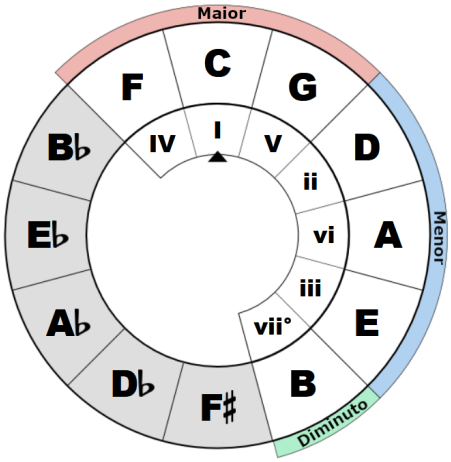
A figura 3 ilustra a escala maior de C em um piano. Nota-se que por ser a escala maior de C, a primeira nota é a nota C e os passos são dados de acordo com o esquema mostrado, que coincidentemente deixa de fora todas as notas sustenidas/bemóis e deixa na escala somente as notas destacadas em vermelho. Devido ao principio da igualdade das oitavas, as escalas geralmente se mantém dentro de uma oitava, podendo ser usadas em oitavas superiores ou inferiores de maneira análoga, se repetindo os mesmos passos.

Os acordes podem ser definidos como três ou mais notas tocando simultaneamente que são ouvidas como uma só, por exemplo, quando um tecladista aperta várias teclas ao mesmo tempo. A "qualidade"de um acorde, como a maioria das coisas em uma harmonia sequencial, depende do contexto no qual ele está inserido, ou seja, a escala com a qual ele se relaciona. No exemplo da escala maior de C utilizado anteriormente, alguns dos acordes aceitos seriam: C maior, F maior, G maior, e alguns dos não aceitos seriam: D maior, G menor e outros. A maior parte dos acordes musicais são tríades, sendo compostos por três notas distintas: a raiz, a terceira e a quinta, onde o posicionamento da terceira e da quinta se dá de acordo com a raiz que é a primeira. No caso do acorde de C maior, as notas que o comporiam seriam: C, E, e G. A figura 4 apresenta o círculo das quintas,uma ferramenta utilizada para saber qual acorde funciona em qual escala.

Existem outras maneiras de se fazer acordes, porém não é necessário se aprofundar nos conceitos desta subseção, pois apenas o seu entendimento básico será necessário para

[3]Open Music Theory. Disponível em: <http://openmusictheory.com/pitches.html>, Acesso em out. 2021

Figura 4 – Círculo das quintas, escala em questão - C maior.[4]



entender o método de avaliação de qualidade, coerência e eficácia da música gerada pela rede neural.

2.2 Redes Neurais Artificiais

De acordo com (GURNEY, 2018), redes neurais artificiais podem ser descrito como uma conexão de múltiplos elementos de processamento, chamados de nós, cuja funcionalidade é vagamente baseada na arquitetura do neurônio biológico, que consegue aprender através dos pesos, que interconectam a rede neural e mudam (aprendem) de acordo com os valores que são utilizados no seu treinamento. Os neurônios biológicos, nos quais se baseiam as redes neurais, podem se conectar a vários outros neurônios formando uma extensa e densamente conectada rede, as conexões entre neurônios são chamadas de sinapses e geralmente ocorrem entre os axônios - parte do neurônio responsável pela condução dos impulsos elétricos com os os dendritos - parte que recebe estes impulsos. Quanto maior a frequência com que uma corrente passa entre um neurônio e outro, mais forte fica a sinapse, por exemplo, um jogador de basquete tem as sinapses cerebrais responsáveis pelos movimentos pertinentes ao seu esporte fortalecidas, fazendo com que certas atividades sejam facilitadas e feitas de modo que pode ser descrito como “automático”.

2.2.1 Neurônio Artificial vs. Neurônio Humano

O neurônio humano, de acordo com (FRIEDMAN et al., 2005), pode vir em diversos tamanhos e formas, porém todos contém os seguintes componentes em comum:

- O corpo do neurônio, também conhecido como soma, que contém o núcleo da célula;
- Dendritos que são extensões da célula que propagam os estímulos eletroquímicos vindos de outros neurônios;

[4]Rand Scullard. Disponível em: <https://randscullard.com/CircleOfFifths/>, Acesso em out. 2021

- Axônios, que são porções alongadas dos neurônios que se estendem para fora do núcleo do neurônio e contêm em suas extremidades terminais nervosos com o intuito de conduzir os impulsos eletroquímicos para fora do neurônio.

Os neurônios têm como funções principais: a recepção, condução e transmissão de sinais, tanto para dentro quanto para fora do sistema nervoso central, onde as informações recebidas são processadas por um sistema de neurônios, que retorna uma resposta.

Os estímulos eletroquímicos recebidos pelos dendritos chegam ao núcleo da célula num lugar chamado cone axionial e, caso o estímulo no cone axial passe de um certo limite, ocorre algo chamado potencial de ação, que é o rápido aumento e diminuição na voltagem (ou potencial de membrana) de uma membrana celular, que causa a propagação da informação recebida pelo neurônio em potencial de ação para outros neurônios ou para o organismo.

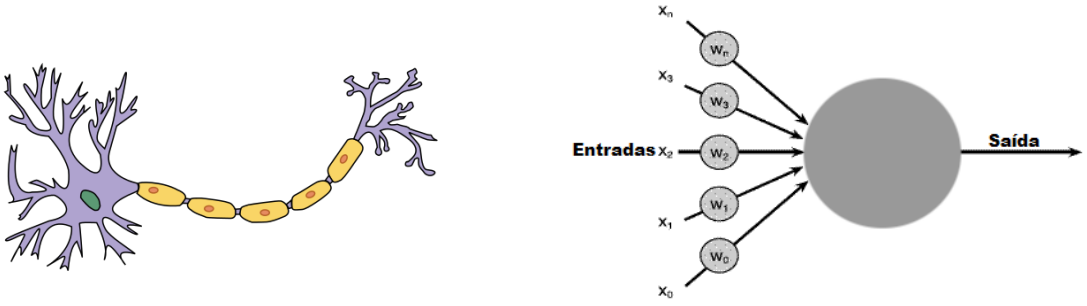
De acordo com (SQUIRE et al., 2008), as zonas de comunicação entre neurônios no sistema nervoso central são chamadas de sinapses e são consideradas essenciais para a transmissão de impulsos nervosos entre neurônios. No sistema nervoso central existem diversos tipos de sinapses, sendo que as características morfológicas e funcionais destes irão variar de acordo com as características e funções dos neurônios que a usam para comunicação.

A neuroplasticidade é um conceito que permeia todos os campos de estudo do cérebro e pode ser vista de diversas maneiras em vários níveis do sistema nervoso, sendo um tipo de plasticidade a plasticidade funcional, que envolve a mudança em aspectos fisiológicos das células nervosas, como por exemplo a frequência de impulsos nervosos, como explicado por (COSTANDI, 2016). Segundo (CITRI; MALENKA, 2007), a plasticidade das sinapses podem variar, sendo tanto reforçadas quanto enfraquecidas pela atividade que exercem e essas mudanças podem durar de milissegundos até ultrapassar dias, se dividindo em duas classes: plasticidade sináptica de curdo período ou plasticidade sináptica de longo período, podendo ser traçado o paralelo com as redes neurais computacionais onde a frequência e volume de dados passados através das conexões entre nós pode ser comparada com as conexões sinápticas, que podem ficar mais fortes ou mais fracas, como exemplificado na rede neural convolucional desenvolvida por (LIN et al., 2018).

O primeiro neurônio artificial foi proposto por Warren McCulloch e Walter Pitts em (MCCULLOCH; PITTS, 1943), denominado *Threshold Logic Unit* (Unidade Lógica Limiar) e que tentava imitar as funcionalidades do neurônio biológico que recebia entradas binárias e retornava na sua saída um valor binário com base no seu limite (*threshold*). Caso a soma das entradas resulte em um valor superior ao limite, o resultado da saída é 1, caso contrário, 0. Desde sua concepção inicial a ideia da formação de redes neurais (integração de mais neurônios) já era presente e, de acordo com (COWAN, 1990), já significava uma notória conquista no ramo da computação, pois estabelecia a validade da criação de modelos formais dos mecanismos cerebrais e a existência de uma teoria

consistente acerca da mente humana. A figura 5 apresenta as representações gráficas de um neurônio biológico e de um neurônio artificial.

Figura 5 – Neurônio biológico e neurônio artificial. [5]



2.2.2 Perceptron

Um modelo de neurônio computacional que revolucionou o ramo da inteligência artificial e do aprendizado de máquinas foi o criado por Frank Rosenblatt em 1958 (ROSENBLATT, 1958), que então tinha a intenção de criar uma máquina capaz de reconhecer imagens e tinha o funcionamento, descrito por (BLOCK, 1962), de maneira a receber um estímulo na primeira camada, chamada camada da retina, que era uma matriz da imagem, onde os pedaços da informação contida na matriz eram enviados adiante no modelo, para a camada dos associadores, através de pulsos, que tinham tempos de atraso que variavam, podendo ter valores positivos ou negativos, dependendo do tipo de informação ali presente. Quando a informação chegava aos associadores, caso a soma dos pulsos ultrapassasse uma determinada barreira, esta informação seria passada, ou para outro ativador ou para as unidades de resposta, que retornaria a classe a qual a imagem pertence.

Na época o desenvolvimento do perceptron foi visto como um grande avanço e a própria equipe responsável por sua criação contribuía para o entusiasmo através de declarações que continham expectativas a respeito do desenvolvimento das capacidades de ler, escrever, falar, andar, ver, se reproduzir e ter consciência da própria existência, como descrito em (NEW..., 1958). Junto com a exposição midiática vieram críticas e controvérsias da comunidade científica, como mostrado em (OLAZARAN, 1996), que também exibe que, apesar das declarações, Rosenblatt sabia das limitações que estavam presentes no perceptron, como falta de mecanismos de pré-processamento, tempo de aprendizado elevado, ineficiência para interpretar ambientes complexos e outras.

As altas expectativas geradas pelo desenvolvimento do perceptron logo foram se esvaindo, até o lançamento do livro *Perceptrons* (MINSKY; PAPERT, 1969), que trazia uma série de provas matemáticas a respeito das forças e das fraquezas do perceptron de

[5] Wikimedia. Disponível em: <https://commons.wikimedia.org/wiki/File:Derived$_N$euron$_s$chema$_w$ith$_n$o$_l$abels.svg.2
[5] Muhammad Abduh. Disponível em: <https://towardsdatascience.com/a-history-of-triggering-artificial-neuron-d1d9853d9fdc/>, Acesso em out. 2021

Rosenblatt, que mudou o rumo da inteligência artificial para um caminho mais pessimista e mostrou as limitações do perceptron como classificador de várias classes e sua efetividade como um classificador binário, que consegue traçar uma linha entre duas classes representadas por regiões bem definidas. Matematicamente o perceptron como classificador binário é definido como:

$$*se\ \Sigma_i\ w_i I_i \geq t\ entao\ y = 1, senao\ (se\ \Sigma_i\ w_i I_i < t)\ entao\ y = 0 \tag{1}$$

onde I_i são as entradas (*inputs*), w_i são os pesos (*weights*) e t é o limite (*threshold*). Um exemplo bem definido no livro de Minsky é a resolução de portões lógicos AND, que, seguindo a lógica do sistema de condições mostrado, precisaria de um limite $t = 2$, onde este retornaria 1 somente no caso da entrada ser 1 em ambas as portas. A mesma ideia pode ser utilizada para as portas lógicas OR, com o limite $t = 1$. O problema para a classificação surge então em portas mais complexas, que é o caso da XOR (ou exclusivo), onde o valor limite para a classificação não pode ser definido de maneira tão clara seguindo a lógica matemática simples do perceptron.

O problema do XOR, como descrito em (MINSKY; PAPERT, 1969), pode ser provado da seguinte maneira. Dada a tabela:

Tabela 2 – Tabela da porta lógica XOR.

I_1	I_2	saída
0	0	0
0	1	1
1	0	1
1	1	0

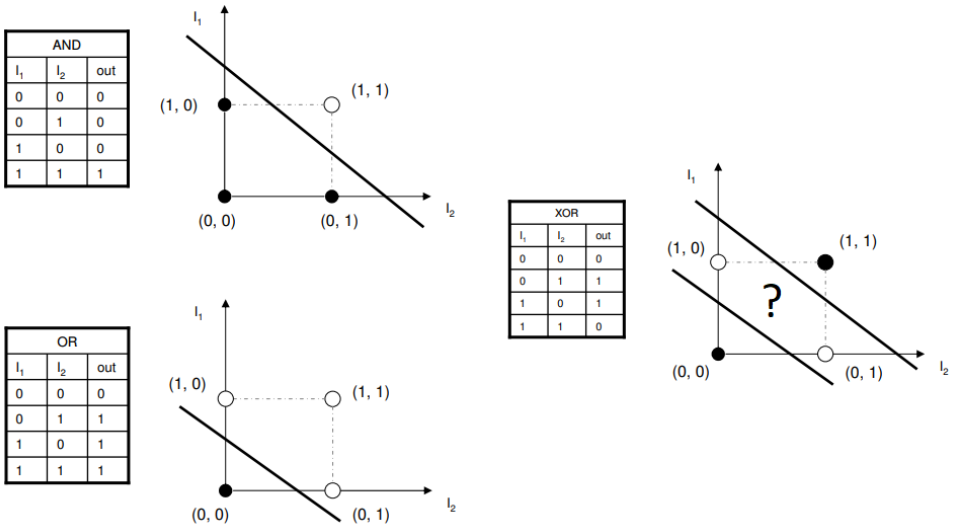
Para o perceptron conseguir definir este problema serão necessários dois pesos, w_1 e w_2, para as duas entradas e um valor limite t, sendo que os seguintes valores causam uma reação (atingem o limite, saída 1): $1 \times w_1 + 0 \times w_2 \geq t$ e $0 \times w_1 + 1 \times w_2 \geq t$ e os seguintes valores não causam uma reação (não atingem o limite, saída 0): $0 \times w_1 + 0 \times w_2 < t$ e $1 \times w_1 + 1 \times w_2 < t$.

Dessa forma temos as contradições: $w_1 \geq t$, $w_2 \geq t$, $0 < t$, $w_1 + w_2 < t$, provando que um perceptron simples (*single-layer*) não pode implementar uma porta lógica XOR, visto que esta não é linearmente separável.

Em (ROSENBLATT, 1961), Rosenblatt explica a convergência do perceptron e se aprofunda na sua habilidade de encontrar resultados, mostrando que o perceptron, quando treinado utilizando duas classes linearmente separáveis, pode convergir em um resultado que se mostra como um hiperplano entre as duas classes, figura 6.

A regra de aprendizado, capacidade de mudar seus pesos de acordo com os acertos, no perceptron, ocorre como descrito em (ROSENBLATT, 1961) e em (FREAN, 1992) da

Figura 6 – Representação gráfica da convergência do perceptron em diferentes portas lógicas.[6]



seguinte forma:

$$\Delta w_i = \alpha(t - o)I_i \tag{2}$$

Onde a mudança de um peso é obtida em função de uma constante positiva α, chamada de *learning rate* (taxa de aprendizagem), de t, que é o valor esperado, de o, o valor da saída obtido e I_i, o valor de entrada observado, dessa forma, a mudança de peso se dá por:

$$w_i \leftarrow w_i + \Delta w_i \tag{3}$$

É possível notar ainda que em $\Delta w_i = \alpha(t - o)I_i$ a parte principal da função está entre parênteses e é representativa da função de erro, que é a diferença entre o valor desejado e o valor obtido e que norteia o perceptron para a sua tomada de decisões, ou seja, quanto menor o resultado a função de erro, menos o perceptron precisa modificar seus pesos, estando assim mais afinado. Esta função é generalizada descrita (MARSLAND, 2014) como:

$$\sum_{k=1}^{N}(y_k - t_k)$$

Onde N é o número de saídas do perceptron, y representa o valor desejado e t o valor obtido.

Em (DAWANI, 2020) são comparados os neurônios de McCulloch e Pitts (MP) e de Rosenblatt (Perceptron), onde é mostrado que a diferença principal entre estes é que o modelo MP simplesmente recebe e soma valores booleanos e retorna 1 ou 0 dependendo

[6] Alexander Hay. Disponível em: <https://alexanderhay2020.github.io/blog/2019/perceptrons/>, Acesso em nov. 2021

do seu valor limite (*threshold*), representado matematicamente:

$$se\ \Sigma_i\ I_i \geq t\ entao\ y = 1, senao\ y = 0 \tag{4}$$

Considerados I como valores de entrada e y como valor de saída. Já o perceptron recebe os valores e os multiplica por uma matriz de pesos para que estes possam ser somados, como já foi representado anteriormente. Essa característica dos pesos do perceptron permite que ele seja capaz de separar linearmente as classes em um conjunto de dados.

2.2.3 Multilayer Perceptron

A ideia de agrupar os neurônios artificiais de maneira a criar um sistema complexo de classificação existe desde o primeiro neurônio artificial (MCCULLOCH; PITTS, 1943), porém, se consolida com a aparição do perceptron pois este traz consigo a vantagem das matrizes de peso, como explicado anteriormente. Atualmente a ideia de *miltilayer perceptron* - perceptron com múltiplas camadas (MLP) representa o modelo de rede neural padrão dentro da área da aprendizagem de máquinas, sendo então uma rede neural *feedforward* composta de, como o próprio nome já diz, diversas camadas (no mínimo três) contendo vários perceptrons.

O aumento das camadas na rede neural artificial torna o campo de estudo mais específico, adentrando então uma subseção da aprendizagem de máquinas chamada aprendizagem profunda (*deep learning*), cujo nome remete justamente a profundidade gerada pelas múltiplas camadas que podem estar contidas na *hidden layer*. De acordo com (GOODFELLOW; BENGIO; COURVILLE, 2016), a aprendizagem profunda trouxe consigo a resolução de problemas onde o objetivo é extrair características abstratas e de alto nível a partir de dados brutos, pois esta permite que o computador consiga construir conceitos complexos (por exemplo o complexo esquema de matrizes de peso em uma rede neural profunda) a partir de conceitos mais simples (neurônio artificial).

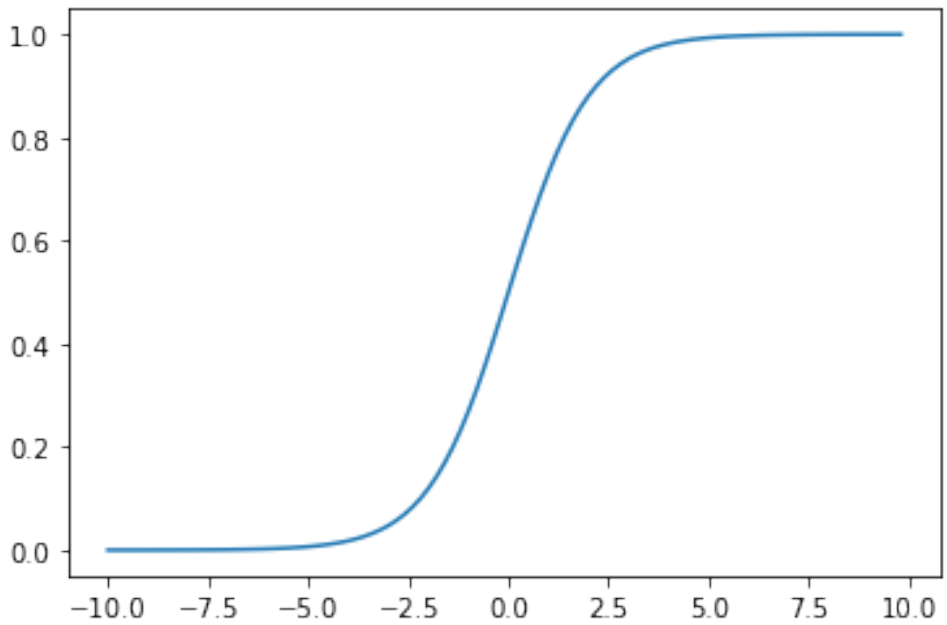
Segundo (TAUD; MAS, 2017), a performance de um modelo MLP não depende somente da escolha dos números de camadas na *hidden layer*, nós e dados de treino, mas também da escolha de parâmetros como a taxa de aprendizagem, controle de momentum, mudanças de pesos e números de iterações, exemplificando que nas redes neurais com apenas uma camada na *hidden layer* a classificação de problemas não lineares não terá alta precisão, mas também MLPs com muitas camadas na *hidden layer* incorrerão em *overfitting* (sobreajuste), que é quando a rede neural ajusta seus pesos tantas vezes que esta fica muito "acostumada"aos dados de treino, podendo ser ineficaz ao resolver um problema real.

A fraqueza do perceptron, como explicado anteriormente, se dá na sua inabilidade de classificação em ambientes onde as classes não podem ser linearmente divididas, como mostrado em (NORIEGA, 2005). As camadas do MLP são: camada de entrada, que re-

cebe os dados, *hidden layer* - camada escondida. De acordo com (TAUD; MAS, 2017), o MLP é uma rede neural *feedforward* com várias camadas onde a informação caminha unidirecionalmente da camada de entrada até a camada de saída, passando pela *hidden layer* que deve ter pelo menos uma camada de perceptrons. Cada conexão entre neurônios em uma MLP tem um peso próprio e todos os perceptrons que estão na mesma camada usam a mesma função de ativação.

As funções de ativação são os métodos com os quais os cálculos são feitos no neurônio, definindo seu valor de saída. Anteriormente, ao explicar o perceptron, pode se dizer que a função de ativação era $peso \times entrada$, porém ao se tratar de grandes redes neurais as funções de ativação têm um papel muito importante, o de introduzir a não-linearidade à rede neural, e são escolhidas de acordo com o tipo de problema que se visa solucionar. E, em (SHARMA; SHARMA; ATHAIYA, 2017), é explicado que uma rede neural profunda que não utilizasse de funções de ativação retornaria em sua saída um polinômio de grau 1, com baixo poder de classificação em dados complexos, portanto se dá necessário o uso das funções de ativação nos nós da rede neural como uma maneira de gerar funções mais complexas de modo a se adaptarem ao conjunto de dados, estas funções são aplicadas nas camadas e geralmente todos os neurônios artificiais de uma camada têm a mesma função de ativação, porém ao longo da rede neural, diferentes camadas podem ter diferentes funções de ativação, essa grande capacidade de combinações de funções diferentes faz com que a escolha da função de ativação correta nas camadas certas seja uma etapa essencial para o bom funcionamento de uma rede neural, como provado em (KARLIK; OLGAC, 2011).

Figura 7 – Representação gráfica da função de ativação Sigmoide, que independentemente do valor de entrada, retorna um número entre 0 e 1.



No multilayer perceptron o aprendizado ocorre através da *backpropagation* (retropropagação), que, explicado de maneira simples, envia o erro no sentido oposto das conexões da rede neural, de modo a afetar seus pesos para que estes, a cada iteração, fiquem cada vez mais coerentes para a classificação dos dados com os quais o modelo é treinado, sendo então parte essencial para o treinamento da rede neural.

De acordo com (MARSLAND, 2014), a ideia por trás da retropropagação é a generalização da mudança de pesos no perceptron, onde a ideia era modificar os pesos baseado na função de erro: $erro = valor_desejado - valor_obtido$, com o intuito de diminuí-la, porém a adição dos pesos extras torna a tarefa mais difícil e exemplifica que um dos melhores métodos para serem aplicados a este tipo de problema é a soma quadrática dos erros:

$$E(t, y) = \frac{1}{2} \sum_{k=1}^{N} (y_k - t_k)^2 \tag{5}$$

Caso seja feita a diferenciação da função apresentada, pode se obter o gradiente da função, que é a direção para qual a função se aumenta ao máximo, dessa forma, como o desejo é diminuir o erro, é preciso seguir a direção oposta ao gradiente, dessa forma "descendo" a função (Figura 8). Mudada então a função de erro, o aprendizado só pode ocorrer de uma maneira, a alteração dos pesos, visto que a saída de uma rede neural depende de três variáveis: os valores de entrada, as funções de ativação dos nós e os pesos da rede neural, sendo que o único que pode ser mutável são os pesos, então, aplicando-se a regra da cadeia e se calculando os gradientes aos pesos dos neurônios artificiais, temos a regra de mudança de peso:

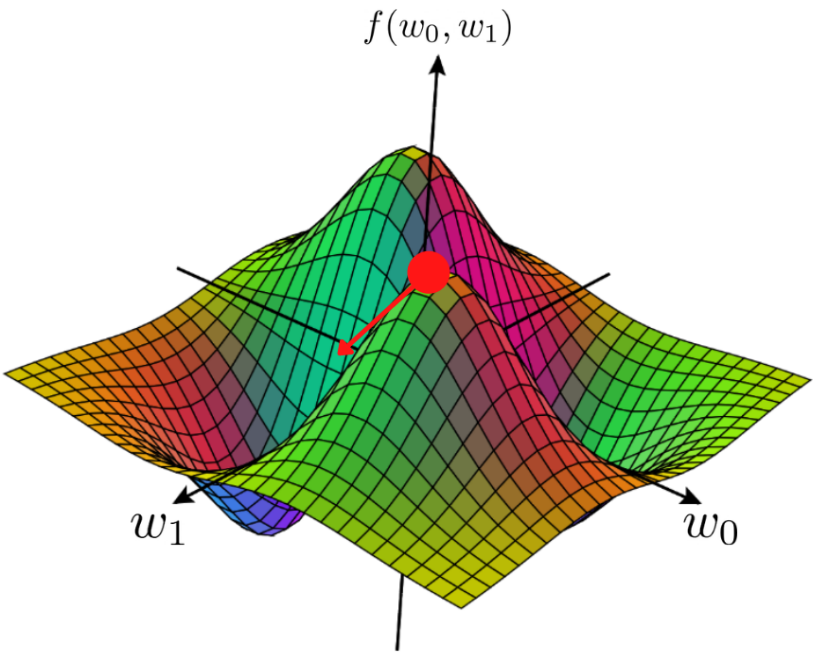
$$w_{ik} \leftarrow +\eta(t_k - y_k)X_i \tag{6}$$

Que é parecida com a regra de mudança de peso no perceptron simples, com a diferença que a regra da cadeia faz com que todos os pesos contribuam entre si para a mudança.

2.2.4 Redes Neurais Recorrentes

O intuito das redes neurais, como explicado anteriormente, está em simular o cérebro humano através de representação matemática e computacional, sendo que uma característica importante do cérebro humano é a sua capacidade de se lembrar de eventos ocorridos. Pensando nos problemas que envolviam a necessidade de memória (ou recorrência de informação), foram criadas as redes neurais recorrentes, que se diferenciam das redes neurais *feedforward* pois as recorrentes possuem a capacidade de armazenar algumas informações importantes para o contexto e as enviarem adiante, sendo então sistemas dinâmicos com representações de estado temporal que podem ser aplicadas em muitos modelos

Figura 8 – Exemplo do ajuste de pesos na direção inversa ao gradiente.



e aplicações de processamento temporal (DU; SWAMY, 2013), e que já foram aplicadas em modelos para: previsões financeiras, tradução computacional, reconhecimento de fala, previsão de influxos de água natural e outros.

Um modelo considerado essencial na área das redes neurais recorrentes é modelo criado por Jordan (JORDAN, 1997) (JORDAN, 1986), que entendia a importância da sequência das informações para o processamento de dados e que propõe sua teoria da ordem serial, que dizia que as ações são produzidas em um contexto temporal e são influenciadas pelas ações próximas a ela, então dada a sequência de ações $x_1, x_2, ...x_r$, produzidas em um determinado plano p que se mantém constante durante a produção da sequência, podendo ser um vetor, virando um conjunto de planos no caso do sistema gerar múltiplas sequências. Os valores da ordem serial não surge de conexões diretas entre as ações, mas sim de duas funções que são calculadas em cada passo do tempo: f, que determina a saída de x_n no tempo n,

$$x_n = f(s_n, p) \tag{7}$$

e uma função g, que determina o estado s_{n+1},

$$s_{n+1} = g(S_N, p) \tag{8}$$

onde ambas dependem do vetor dos planos e do vetor do estado atual (s). Na teoria, o resultado da função f deve surgir através das associações entre o vetor do estado

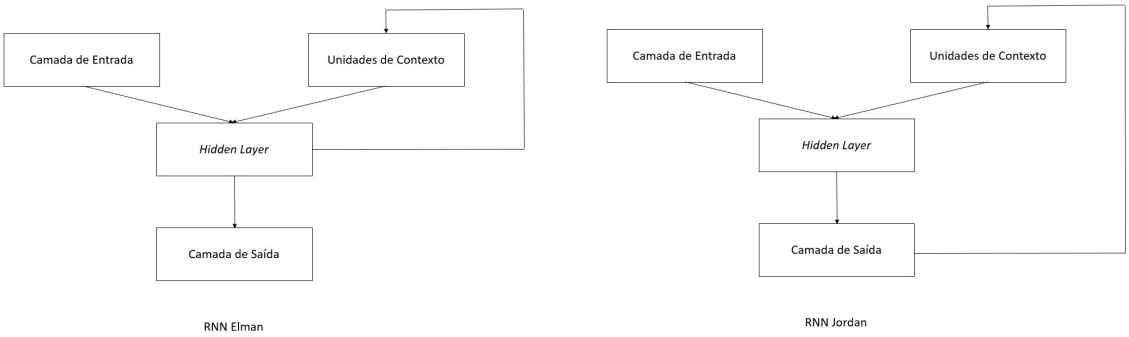
e o vetor do plano em relação ao vetor de saída, pois estados e planos similares resultam saídas similares. O valor obtido em f é implementado no modelo através dos pesos das conexões entre os valores de entrada (planos), de estado e de saída, incluindo a *hidden layer*, que permite o uso das funções não lineares. O outro elemento do modelo de Jordan é a função do próximo estado, que é implementada com conexões recorrentes das unidades de estado para com elas mesmas e das unidades de saída para as unidades de estado, permitindo que o valor atual do estado seja uma combinação entre o valor anterior do estado e o valor anterior da saída. De maneira simples, a aprendizagem no modelo Jordan ocorre como um processo de correção de erros, onde os pesos da rede neural são ajustados de acordo com a diferença entre a saída atual da rede e a saída desejada, mas se difere de uma *feedforward* pois a função do próximo estado é que fornece o vetor de estado, que varia com o tempo, e é injetada na hidden layer junto com as unidades de plano.

Outro modelo essencial para se compreender as redes neurais recorrentes é modelo criado por Elman (ELMAN, 1990), que se parece muito com o modelo criado por Jordan, com a diferença de que não existem unidades de estado, mas sim de contexto que não se conectam consigo mesmas e onde as unidades que são responsáveis pela recorrência são aquelas da *hidden layer*, que enviam suas informações para as unidades de contexto. O aprendizado de um modelo recorrente de Elman é exemplificado por (SMAGT; KROSE, 1995) como:

1. As unidades de contexto recebem o valor 0 em $t = 1$;
2. O padrão x^t é estabelecido e os cálculos são feitos;
3. A regra de aprendizado baseada em retropropagação pode ser aplicada, pois agora a *hidden layer* tem valores que podem ser enviados a camada de contexto;
4. $t \leftarrow +1$; tempo é estabelecido como 2.

Onde as unidades de contexto estão no passo t, logo, sempre recebem os valores da *hidden layer* que estão no passo $t - 1$.

Figura 9 – Elman x Jordan. [7]



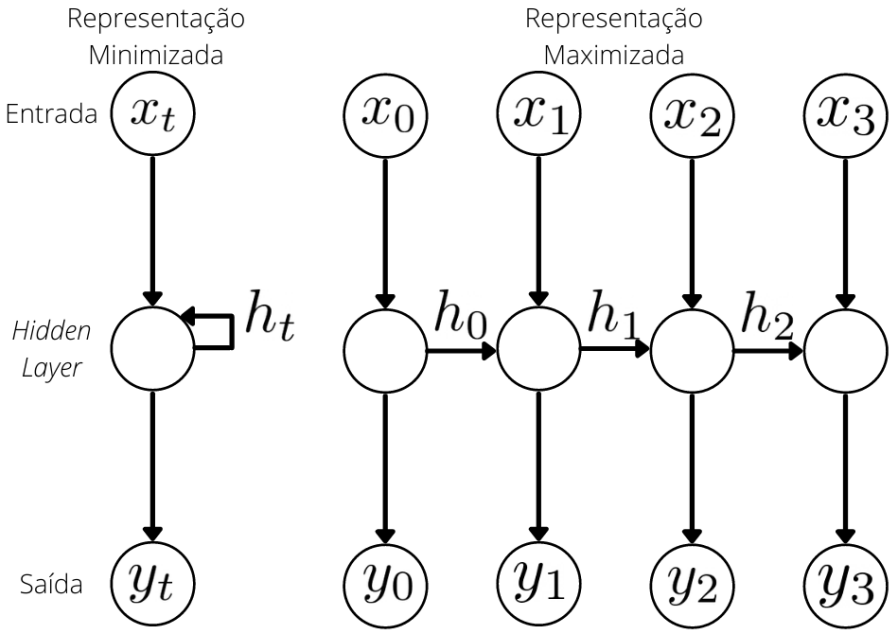
Um exemplo que pode ser utilizado para se explicar as redes neurais recorrentes é o seguinte: na língua francesa, a frase “je mange”pode ser traduzida como “eu como”e a frase

“il mange”pode ser traduzida como “ele come”, pode se notar então que a palavra “mange”, dependendo do sujeito, pode ser traduzida tanto como “como”ou “come”, dessa forma uma dúvida pode surgir: supondo que haja uma rede neural *feedforward* treinada para traduzir palavras do francês para o português e que esta recebesse as palavras na ordem em que elas são lidas, como esta conseguiria manter a coesão do verbo “mange”sem saber qual palavra o antecede? A solução para este problema está nas redes neurais recorrentes, que aplicam a recorrência aos seus neurônios e são frequentemente utilizadas, como dito anteriormente, em aplicações de tradução computacional.

A Figura 10 apresenta a recorrência em uma rede neural, que quando aplicada aos neurônios age de maneira a conectar a rede neural consigo mesma através do tempo. Usando como exemplo o perceptron apresentado no começo da seção, este poderia ser considerado como apenas um passo no tempo de uma rede neural recorrente, e assim também poderia ser considerada a rede neural *feedforward*, onde estes enviariam sinais relacionados ao resultado de seus cálculos ao próximo passo no tempo, caso não fossem eles o último passo, e receberiam informações do passo anterior, caso não fossem eles o primeiro passo, transformando a equação do resultado em uma equação em função do passo no tempo e da informação recebida no passo anterior:

$$y_t = f(x_t,\ h_{t-1}) \tag{9}$$

Figura 10 – Representação gráfica de uma rede neural recorrente simples.



E o valor que é passado adiante é:

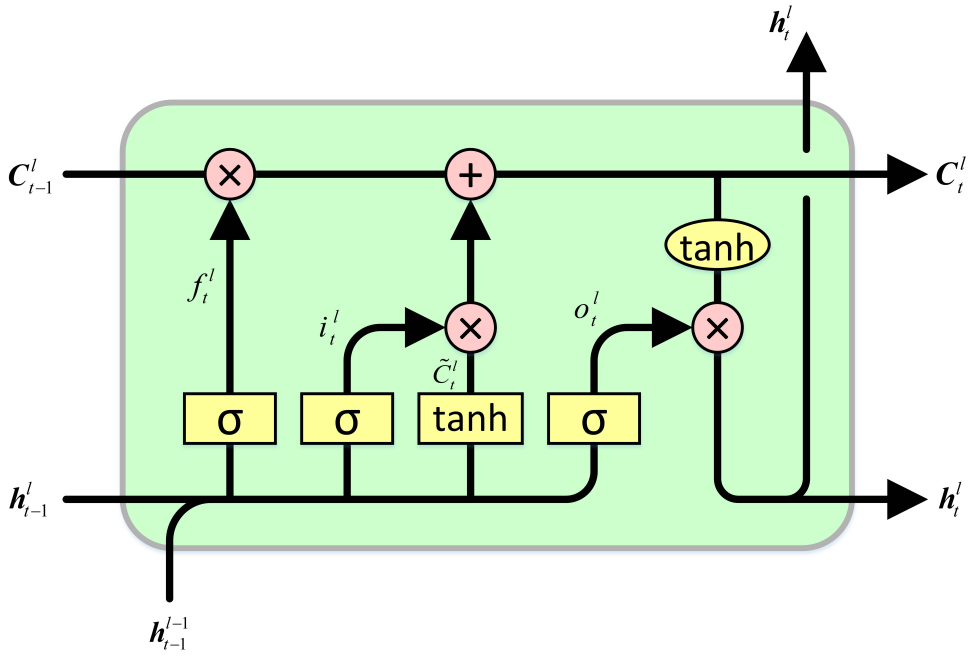
$$h_t = f_w(x_t, h_{t-1}) \tag{10}$$

Onde f_w é uma função de pesos, que é aprendida pela rede neural e que está em função da entrada x_t e do estado h_t anterior.

2.2.5 Redes Neurais Recorrentes Long Short-Term Memory (LSTM)

As redes neurais *Long Short-Term Memory* (LSTM) são um tipo de rede neural recorrente introduzidas primeiramente por Jürgen Schmidhuber e Sepp Hochreiter em (HOCHREITER; SCHMIDHUBER, 1997) pois estes notavam problemas com o uso da retropropagação como método de ajuste de pesos nas redes neurais recorrentes: o problema do *vanishing gradient* (desaparecimento do gradiente) e da *gradient explosion* nos gradientes de longo termo, conseguindo então trazer uma solução parcial para o problema do desaparecimento do gradiente, mas não para o problema da explosão.

Figura 11 – Célula LSTM e seus portões.[8]



O fluxo de dados numa rede neural LSTM ocorre através de unidades chamadas células LSTM, mostradas na Figura 11 que apesar do nome, descrevem o comportamento de toda a *hidden layer* e o funcionamento desta unidade ocorre da seguinte forma: além de receber a informação vinda da rede neural num espaço de tempo anterior, assim como fazem as RNNs, porém a célula LSTM também recebe do espaço anterior do tempo um dado chamado *cell state* (estado da célula), que é uma informação considerada importante

[8] Usuário JM Zhou, Stack Overflow. Disponível em: <https://stackoverflow.com/questions/50488427/what-is-the-architecture-behind-the-keras-lstm-cell

para se manter a coesão no tempo, sendo que esta informação pode se manter afetando os resultados de saída por um tempo muito maior do que nas RNNs.

A célula de estado é controlada pela rede neural LSTM a partir dos *gates* (portões), sendo que em uma célula LSTM básica nós temos três portões, que são representados, segundo (WANG; JIANG, 2015), (ROCKTäSCHEL et al., 2015) e (SMAGULOVA; JAMES, 2019) como: *forget gate* (portão de esquecimento), que decide quais informações precisam de atenção e quais informações podem ser ignoradas, representado por:

$$f_t = \sigma(W_f \times [h_{t-1}, x_t] + b_f) \tag{11}$$

input gate (portão de entrada), que decide quais informações serão atualizadas no *cell state*, representado por:

$$i_t = \sigma(W_i \times [h_{t-1}, x_t] + b_i)\hat{C}_t = \tanh(W_c \times [h_{t-1}, x_1] + b_C \tag{12}$$

output gate (portão de saída), que determina o valor do próximo *hidden state* (h), representado por:

$$o_t = \sigma(W_o[h_{t-1}, x_1] + b_o) \tag{13}$$

Com o valor de *cell state* C_t e saída h_t definidos como:

$$C_t = f_t \times C_{t-1} + i_t * \hat{C}_t h_t = o_t \times \tanh(C_t) \tag{14}$$

Onde σ representa a função sigmóide, tanh é a função tangente hiperbólica, t o local no tempo, x a entrada (*input*), h é o estado, W_f, W_c, W_i e W_o são matrizes de peso, b_f, b_i, b_C e b_o são as tendências (*biases*).

2.3 Formato MIDI

O formato MIDI, como descrito na introdução, foi criado para servir como uma linguagem geral para que instrumentos feitos por diferentes companhias pudessem se comunicar entre si, sendo um indicador de eventos, ou seja, não sinaliza sons propriamente ditos, mas sim movimentos como o apertar de teclas, trazendo consigo vantagens como a portabilidade, pois um arquivo MIDI não pesa tanto quanto um mp3, por exemplo, e a facilidade de manipulação de notas e tons sem a necessidade de se repetir uma gravação instrumental, capacidade esta que foi tão aproveitada que hoje em dia há a possibilidade de criação musical direto em computadores, sem a necessidade de qualquer instrumento através de softwares de criação musical como *FL Studio*, *Ableton*, *LogicPro* e outros. De acordo com (LOY, 1985), as especificações do código por trás do formato MIDI consistiam de três elementos: canais, modos e comandos (no presente trabalho chamados de

mensagens), que serão discutidos nas próximas subsessões.

As informações exibidas nesta seção serão utilizadas durante a segunda parte do trabalho como maneiras de se obter as informações que compõem a música que será "dissecada" e transformada em tensores para o processamento da rede neural, bem como para a composição do produto final gerado pela rede neural, sendo necessárias para a montagem das notas criadas a partir da música base, permitindo que seja feita a decodificação das informações previstas pela rede neural em melodias também no formato MIDI.

2.3.1 Canais MIDI

Os canais MIDI são os caminhos de comunicação pelo qual as informações dos diferentes dispositivos são recebidos e interpretados, por exemplo, na criação de uma música MIDI por computador, caso haja uma guitarra e um baixo, cada instrumento terá sua composição feita em um canal MIDI diferente, essa separação virtual traz vantagens ao formato MIDI pois permite que a mixagem e a masterização sejam feitas individualmente e também permite a troca de timbres, fazendo possível que instrumentos diferentes possam tocar usando composições de outros canais, e no caso do presente trabalho, permite que seja feita a separação de informações para o pré-processamento de maneira mais fácil. Um arquivo MIDI tem 16 canais, que são descritos por (LOY, 1985) como fornecedores de controles de multissintetizador dentro de uma rede MIDI.

2.3.2 Modos e Estados MIDI

Os modos MIDI se dividem em três: *omni*, *poly* e *mono* e os estados MIDI são quatro, compostos dos três modos citados e são descritos como: Estado 1 - *omni on, poly*, onde as mensagens podem ser recebidas em qualquer canal e as notas podem ser executadas polifonicamente, Estado 2 - *omin on, mono*, onde as mensagens de podem ser recebidas em qualquer canal e as notas são executadas monifonicamente, Estado 3 - *omni off, poly*, onde as mensagens são transmitidas somente por um canal, e as notas podem ser executadas polifonicamente, Estado 4 - *omni off, mono*, permitindo somente uma nota por vez através de apenas um canal, também conhecido simplesmente como estado mono. Estes modos servem para gerenciar a distribuição de vozes musicais e de canais.

2.3.3 Mensagens (Comandos) MIDI

Mensagens MIDI são as informações passadas dentro do arquivo MIDI que servem para coordenar a comunicação e, de acordo com (HASS, 2020), podem ser divididos entre duas categorias principais: mensagens de canal e mensagens de sistema. Os mensagens de canal podem ser:

- Mensagens de canal de voz, que são considerados os mais básicos no formato MIDI e que na maioria dos casos servem para sinalizarem os eventos performados por

instrumentos através de algum dos dezesseis canais MIDI, e são compostos por um byte de status, que indica qual evento ocorre e vai de 128 até 255, seguido por dois bytes de dados que podem ir de 0 até 127;

- Mensagens do modo do canal, que são representados por bytes de 121 até 127 e modificam a interação do instrumento com todos os dezesseis canais MIDI que lhe são disponíveis.

Os mensagens de sistema são:

- Mensagens comuns do sistema, que são, assim como o nome diz, comuns a todos os instrumentos e canais midi e afetam todo o sistema MIDI, requerido que todos os dispositivos conectados a ele enviem uma resposta de volta;

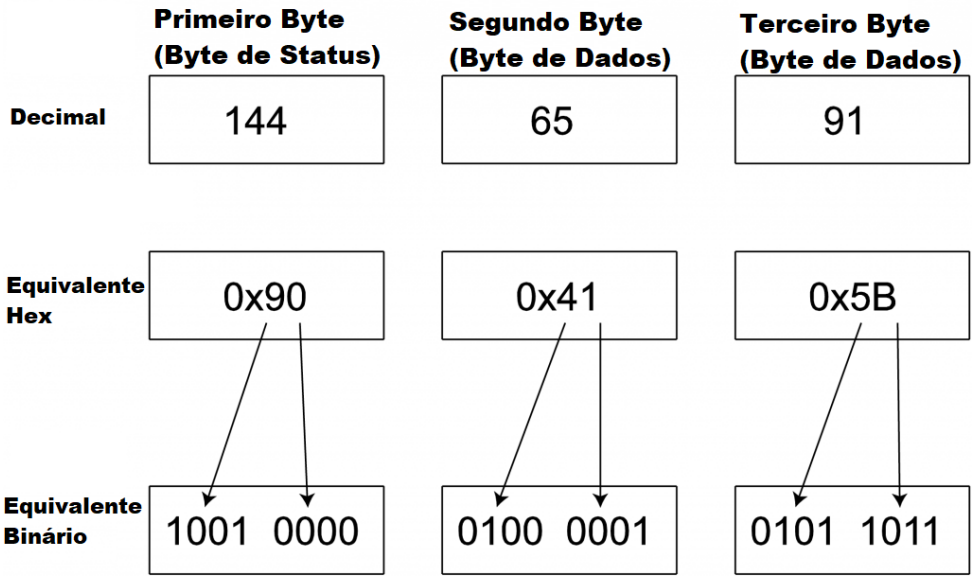
- Mensagens em tempo real do sistema, que são um conjunto de mensagens considerado crucial e que se refere a natureza temporal da reprodução da música, são comandos que afetam o sistema inteiro e geralmente servem para funções de sincronização;

- Mensagens exclusivas do sistema, também chamadas de Sysex, são as mensagens criadas para transmitir informações que dizem a respeito de funções específicas em um hardware com capacidades MIDI, e são criadas pelas empresas que fabricam o produto.

Na Figura 12, pode se ver uma mensagem de *Note On*, onde o primeiro byte representa o tipo de mensagem (primeiro nibble) e seu canal (segundo nibble), o segundo byte representa a nota (F5) e o terceiro byte representa a velocidade (força com a qual se pressiona a tecla, que varia de 0 até 127).

[9]Digital Music Sound. Disponível em: <http://digitalsoundandmusic.com/6-1-5-a-closer-look-at-midi-messages/>, Acesso em nov. 2021

Figura 12 – Representação gráfica de uma mensagem MIDI.[9]



3 METODOLOGIA

Este capítulo visa descrever, com base nas técnicas e informações previamente apresentadas, o processo através do qual foi abordado o desenvolvimento da proposta do presente trabalho em sua realização, podendo ser dividido em quatro partes: coleta de dados, pré-processamento, modelagem da rede neural e pós-processamento.

3.1 Ferramentas Utilizadas

As ferramentas utilizadas na prática deste trabalho de conclusão de curso foram:

3.1.1 Hardware

- *Notebook Lenovo* ano 2019;
- Processador *Core it 8th gen*;
- Placa de vídeo *NVIDIA MX 110*;
- Memória RAM 8GB;
- HD 1TB;
- Sistema Operacional *Linux kali 5.18.0-kali7-amd64*.

3.1.2 Software

3.1.2.1 Linguagem de Programação Python

Esta linguagem de programação foi escolhida por ser de uso geral e demonstrar uma grande variedade de bibliotecas para as duas colunas principais que sustentam este trabalho: análise de dados e inteligência artificial, além de ser concisa e consistente, fornecendo um código mais simples de ser planejado e escrito.

As bibliotecas utilizadas na realização do trabalho (fora as bibliotecas já inclusas na linguagem de programação) foram:

- Music21: biblioteca que contém um conjunto de ferramentas para a musicologia aliada à computação. Esta biblioteca é utilizada para responder questões relacionadas à musicologia, estudar grandes conjuntos de dados musicais, geração musical, edição de notação musical e composição musical. O projeto para essa biblioteca foi iniciado no MIT - *Massachusetts Institute of Technology* e possui a filosofia "ouça mais rápido", que remete às capacidades de análise de dados da biblioteca e significa que o usuário da biblioteca pode encontrar pontos interessantes e informações gerais sobre o perfil musical de uma determinada música ou grupo de arquivos musicais

sem precisar necessariamente ouvi-los, mas apenas utilizando as funcionalidades da biblioteca.

- TensorFlow (via Keras): O TensorFlow é uma biblioteca open-source de aprendizado de máquina e inteligência artificial desenvolvida pela Google e permite que desenvolvedores criem fluxogramas de dados, estruturas que mostram como os dados se movem através de um grafo, onde cada nó reprenta uma operação matemática, e cada conexão entre nós é um array multidimensional, chamado de tensor. Na linguagem de programação escolhida, Python, o TensforFlow se manifesta através dos objetos python e as aplicações do TensorFlow se transformam em aplicações Python, porém os cálculos matemáticos em si são realizados através de binários C++ compilados.

  A biblioteca Keras, por sua vez, é uma API do TensorFlow, que permite a manipulação e criação de modelos complexos com uma sintaxe mais simplificada que segue as melhores práticas para a redução da carga cognitiva. Ajudou a produzir o presente trabalho através da criação modelos complexos que envolvem tensores do TensorFlow são todos encapsulados em objetos da API Keras, dividindo o que pode se tornar uma complexa rede neural em blocos de componentes Keras através da linguagem Python.

3.1.2.2 MuseScore

Além da biblioteca music21 mencionada anteriormente, durante a realização deste trabalho também foi utilizado o software MuseScore, um software livre e *open-source* de edição musical que tem o propósito de criar partituras musicais de alta qualidade com interpretadores e reprodutores musicais automáticos e com suporte para a entrada e saída de arquivos do tipo MIDI, formato escolhido para ser utilizados no presente trabalho.

3.2 Coleta de Dados

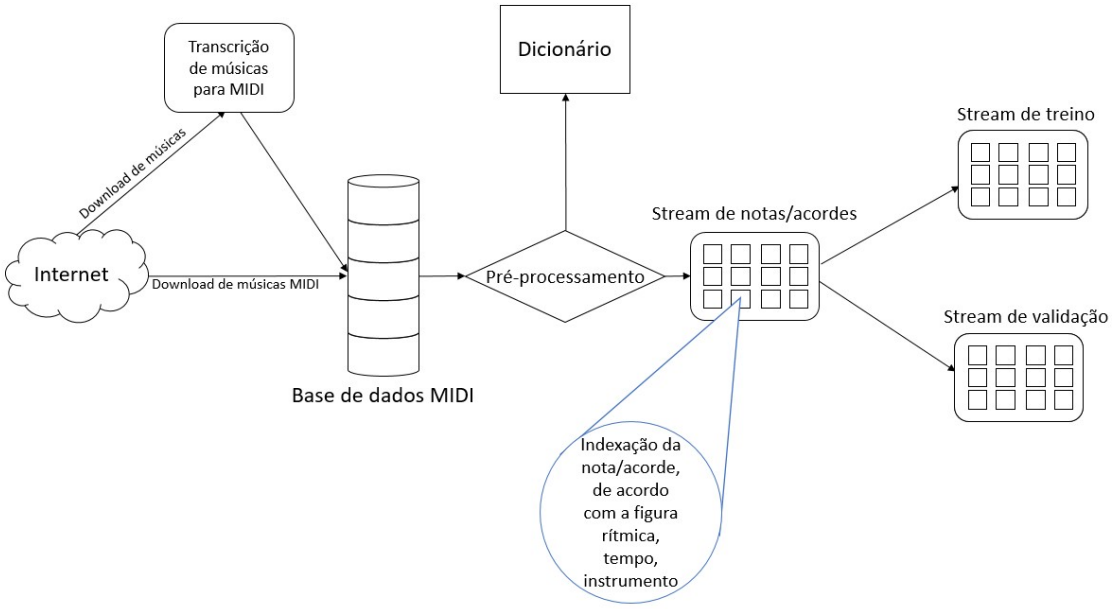
Os dados referentes a arquivos MIDI são músicas de jogos de videogame antigos, preferencialmente aqueles descritos em (COLLINS, 2005) como pertencentes a segunda parte da década de 1980 até o final da década de 1990, bem como jogos cujas trilhas sonoras foram codificadas em formato MIDI por fãs e postados na internet, em sites como www.vgmusic.com e www.khinsider.com. Também poderão ser coletadas as transcrições musicais feitas por fãs para formato de partitura e, com o auxílio de programas como o *MuseScore* e o *LilyPond*, estas podem ser então transformadas no formato MIDI e ainda, de maneira menos fidedigna, podem ser convertidos arquivos musicais em mp3 para o formato MIDI com a ajuda de *converters* na internet. Esta etapa é demonstrada em parte do diagrama da Figura 13.

3.3 Pré-Processamento dos Arquivos MIDI

O pré-processamento para aplicações que envolvem *machine learning* e redes neurais é uma etapa crucial, pois envolve os ajustes necessários em dados do mundo real (no caso do presente trabalho, aquivos MIDI), que podem conter informações faltosas, desnecessárias, não-normalizadas e impróprias para serem trabalhadas pelo algoritmo.

Uma vez coletada a base de dados contendo os arquivos utilizados no treinamento e no teste da rede neural, serão necessárias as etapas de pré-processamento e extração das informações que sejam úteis, sendo utilizadas bibliotecas de programação específicas para o manuseio de arquivos MIDI para a coleta dos dados descritos na subseção 2.3.3, facilitando a detecção de notas e acordes em um arquivo. A figura 13 descreve a etapa de coleta de dados e de pré-processamento no presente trabalho, onde pode se verificar que o pré-processamento gera dois produtos que são diferentes mas complementares: um deles é o dicionário, que é a relação índice-informação de todos os comandos MIDI em todas as músicas da base de dados, e o outro produto é a stream de notas/acordes que é a substituição das informações obtidas pelo algoritmo pelos índices do dicionário, que é a troca dos dados categóricos por numéricos, bem como a normalização dos dados ali presentes, para que não haja discrepância entre valores e evitando o risco de desbalanceamento da rede neural.

Figura 13 – Representação gráfica do pré-processamento de dados.



A stream de notas/acordes é um arquivo único contendo a sequência dos índices dos eventos capturados pelo algoritmo de pré-processamento em todas as músicas, que então é dividido em duas partes: a primeira parte é o stream de treino, que é utilizada

para treinar a rede neural e a segunda parte é o stream de validação, que é menor que o stream de treino, e é utilizada para testar a habilidade da rede neural em si, sem avaliar o resultado musical obtido, testando somente a capacidade de previsão da rede neural e serve para evitar que a rede se acostume demais aos dados de treinamento (*overfitting*).

A parte de pré-processamento de dados também é bastante útil pois ajuda na visualização de dados referentes as redes neurais, que podem facilitar a percepção de dados faltosos, discrepantes e tendenciosos dentro da stream de dados, permitindo também a elaboração de gráficos, tabelas e quadros de dados para a obtenção de estatísticas.

Na prática, o pré-processamento segue os seguintes passos:

1. Músicas são baixadas do site vgmusic.com e todas são salvas na mesma pasta, que será alvo inicial para o pré-processamento;

2. As músicas são carregadas para dentro da memória do computador através da um script Python, que escolhe todos os arquivos com final '.mid' que estão dentro da pasta citada no passo anterior, os transforma em objetos Score do Music21, que facilita a extração de informações dos mesmos e os salva todos em um array;

3. O array gerado no passo anterior é percorrido novamente e cada músia dentro dele é transposta, visto que as músicas vêm em chaves diferentes, então deve haver uma planificação dessas notas para que as músicas com chaves maiores sejam transpostas para a escala de C Maior(Dó Maior) e as músicas com chaves menores sejam transpostas para a escala de A Menor (Lá Menor). Este passo foi realizado da seguinte forma: o array gerado no passo anterior foi percorrido e a chave musical foi extraída com o auxílio da biblioteca Music21, então, ainda com o auxílio da Music21, se calcula a distância entre a chave atual e a chave da escala desejada, levando em consideração se a chave atual é maior ou menor, pois caso seja maior, a distância é entre a chave atual e a chave de C (dó) (transpondo para a escala de C Maior), caso seja menor, a distância é entre a chave atual e a chave de A (Lá) (transpondo para a escala de A Menor). Após calculada a distância, a música, com o auxílio da biblioteca Music21, é transposta pelo valor correspondente à distância e adicionado a um novo array. A escolha das escalas C Maior e A Menor se dá pois elas são equivalentes (C Maior: C-D-E-F-G-A-B, A Menor: A-B-C-D-E-F-G), porém com ordem diferente;

4. Uma vez que a base de dados está pronta para ser manipulada ocorre a codificação, que transforma cada arquivo presente no array que resulta do estado anterior em strings para que estas possam ser posteriormente analisadas e indexadas. A codificação das músicas se dá da seguinte forma: o array é percorrido música por música (para cada música uma string nova é criada), onde cada música é percorrida evento

por evento, sendo que cada evento é um acontecimento na "linha do tempo" do arquivo MIDI. Cada evento é checado e transformado em um símbolo, sendo que o evento que interessa para o presente trabalho pode ser de três tipos: *Rest* (pausa), *Chord* (acorde) e *Note* (nota), caso o evento pertença a qualquer um desses tipos, seu valor (botões pressionados) e duração são obtidos e então transpostos para o símbolo, que contém três informações: '<TIPO>~<VALOR>~<DURAÇÃO>' (no caso de pausa, o <VALOR> é equivalente a 'p', pois a pausa não tem valor) e após a criação deste símbolo, o caractere '|' é adicionado ao final, pois este funciona como uma divisão entre cada um dos componentes únicos (eventos codificados) e então o símbolo é adicionado à string total da música que está sendo pecorrida atualmente.

Após uma música ter sido completamente codificada em string, uma string de divisão é adicionada ao final da string da música. Essa string de divisão contém um símbolo próprio que é repetido por N vezes (" 'ç|' × N"), onde o valor de N corresponde ao tamanho do lote que será analisado pela rede neural (64) e serve para indicar quando uma música acaba e outra começa. Após todas as músicas terem sido percorridas, as strings geradas são somadas umas as outras e salvas em um arquivo de texto (stream de dados), que posteriormente será utilizado para o treinamento da rede neural. Abaixo está um exemplo de pedaço de stream de dados contendo 10 eventos:

```
Chord~A-4 A-5~1.25|Chord~A-4|A-5~1.25|Rest~p~4.0|
Rest~p~4.0|Rest~p~4.0|Note~B-~0.25|Rest~p~0.25|
Note~B-~0.25|Rest~p~0.25|Rest~p~0.25|
```

5. Após a criação da stream de dados em um arquivo de texto único, este arquivo é lido pelo programa e carregado em uma string. A string então é dividida em um array de strings baseado no caractere '|', o array é percorrido e indexado em um dicionário python, onde cada string que correspodne a um evento é atribuido a um valor inteiro. Esse dicionário python é então transformado em um arquivo JSON e salvo para uso posterior. Exemplo de pedaço do arquivo JSON gerado:

```
{
"Note~A~0.25": 0,
"Rest~p~4.0": 1,
...
"Chord~G4 A4~0.25": 177,
"Note~B~5/12": 178
}
```

A divisão entre stream de treino e de inferência demonstrada na Figura 13 é realizada, na prática, de maneira automática pela biblioteca Scikit-learn.

O tipo de pré-processamento explicado até aqui foi utilizado na prática da maneira descrita na seção 4.1 (dados de Tipo 1), porém se mostrou ineficiente devido ao uso dos acordes, então foi remodelado de maneira a não incluí-los. A diferença entre os dados descritos no capítulo 4 é que, apesar de ambas seguirem a sequência mostrada na seção 3.3, os dados de Tipo 1 incluem os acordes e os dados de Tipo 2 incluem apenas as Notas.

3.4 Modelagem da Rede Neural

O tipo de rede neural LSTM que será utilizada no presente trabalho é do tipo *many-to-one* (muitas para um), onde a rede recebe uma quantidade N de comandos MIDI pré-processados correspondente a N espaços anteriores no tempo e tenta prever a nota que vem em sequência. Para metodologia do presente trabalho, a etapa de modelagem da rede neural se divide em duas etapas: treinamento e inferência, que serão explicadas nas subseções a seguir.

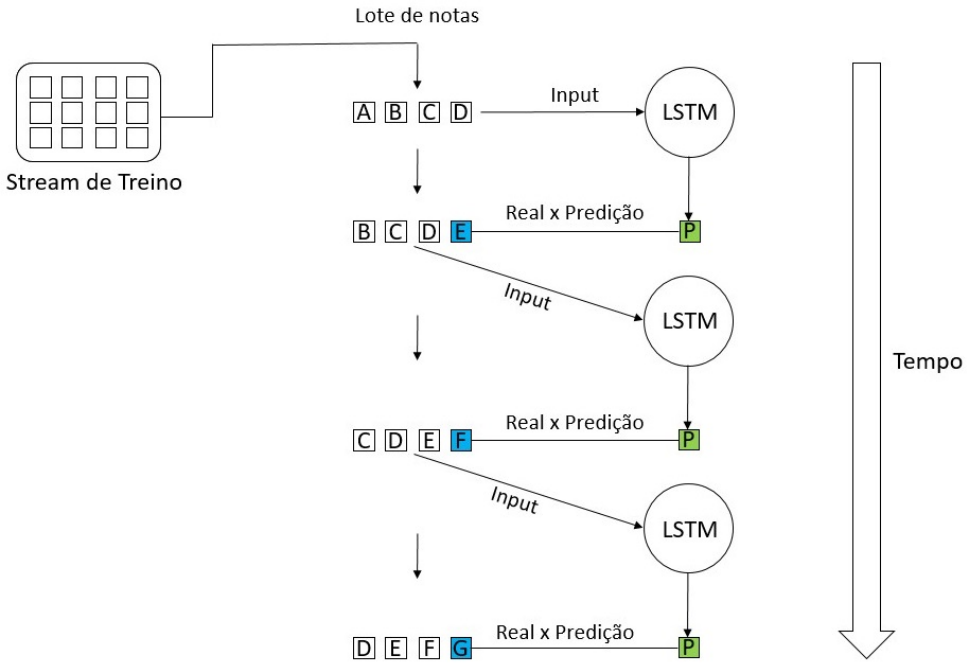
Na prática, o modelo criado em Python, com o auxílio da biblitoeca Keras, é do tipo sequencial com 4 camadas, uma contendo a camada de entrada, que recebe um array de tamanho igual ao tamanho de itens no dicionário criado pelo pré-processamento, a segunda é uma camada do tipo LSTM com 256 nós internos que recebe os dados da camada de entrada, a terceira é uma camada do tipo Dropout, que seleciona aleatoriamente, em cada passo do processamento, 20% dos neurônios de entrada e os ignora propositalmente para evitar que ocorra overfitting e a última camada é a camada de saída, que é do tipo Denso, onde cada um dos neurônios está conectados com todos os neurônios da camada anterior e possui uma função de ativação do tipo softmax, que retorna um array com o mesmo tamanho do array de entrada (número de itens no dicionário) contendo uma distribuição de probabilidades em cada uma das posições, onde a posição com o maior número é a posição do dicionário mais provável de ser executada de acordo com os 64 valores anteriores. O modelo é então compilado com o auxílio da função 'compile' pertencente ao objeto 'Model' do Keras, com a função de perda do tipo entropia cruzada categórica esparsa, utilizada para a comparação de predições em arrays, com o *optimizer* 'Adam', que pertence à biblioteca Keras e se baseia no método estocástico de gradiente descendente, sendo que a métrica utilizada para julgar o desempenho do modelo é a acurácia.

3.4.1 Treinamento e Validação da Rede Neural

O treinamento de uma rede neural LSTM *many-to-one* ocorre da seguinte maneira: os dados pré-processados são selecionados em lotes de N eventos anteriores que são fornecidos para a rede, que com base nestas informações tenta prever qual é o próximo valor, como demonstrado na Figura 14:

Uma vez previsto o próximo valor pela rede neural, há a comparação entre o valor real, que é adicionado ao final do lote (o primeiro evento do lote não é mais utilizado, mantendo-se o tamanho N de eventos por lote) e a predição para se avaliar a precisão da rede neural e para que os pesos das células LSTM sejam ajustados. A partir daí ocorre a repetição do treinamento, que é a tentativa de previsão com os pesos ajustados e a comparação com o próximo evento.

Figura 14 – Treinamento da Rede LSTM.



A validação da rede neural ocorre da mesma maneira que o treinamento, a diferença é que a stream de validação é utilizada e as métricas obtidas de perda e precisão na validação são consideradas as oficiais, visto que quando a rede não é validada ela pode se mostrar viciada nos dados de treinamento (*overfitting*).

Na prática, o treinamento da rede neural ocorre da seguinte maneira:

1. São criados dois arrays vazios, um que vai armazenar as entradas e outro que vai armazenar as saídas desejadas que serão comparadas com as saídas obtidas pela rede neural.

2. O arquivo de texto contendo a stream de dados gerado no pré-processamento é carregado para a memória do computador e é transformado em um array de strings através do método *split*, que a separa onde quer que haja um caractere '|' (caractere divisor previamente escolhido).

3. As músicas são então convertidas de strings para valores inteiros, onde cada valor é percorrido e utilizado como chave do dicionário, tendo seu valor trocado pelo valor

inteiro que está no dicionário, retornando a stream de dados em formato inteiro dentro de um array.

4. As entradas são de arrays de inteiros de tamanho 64 e a saída é um valor único inteiro. Ou seja, são selecionados recortes dentro do array fornecido no passo anterior para os valores de entrada e de saída, por exemplo: no primeiro conjunto entradas/saída, o array inteiro terá o sub-array de entrada selecionado da posição 0 até 63 e a saída será o item que ocupa a posição 64, o segundo conjunto entradas/saída terá o sub-array de entradas selecionadas entre as posições 1 e 64 e a saída será o item que ocupa a posição 65, e assim por diante em todo o array de inteiros. Os sub-arrays de tamanho 64 são salvos no array de entradas citado no passo 1, que vira um array de arrays, e as são salvas no array de saídas também citado no passo 1.

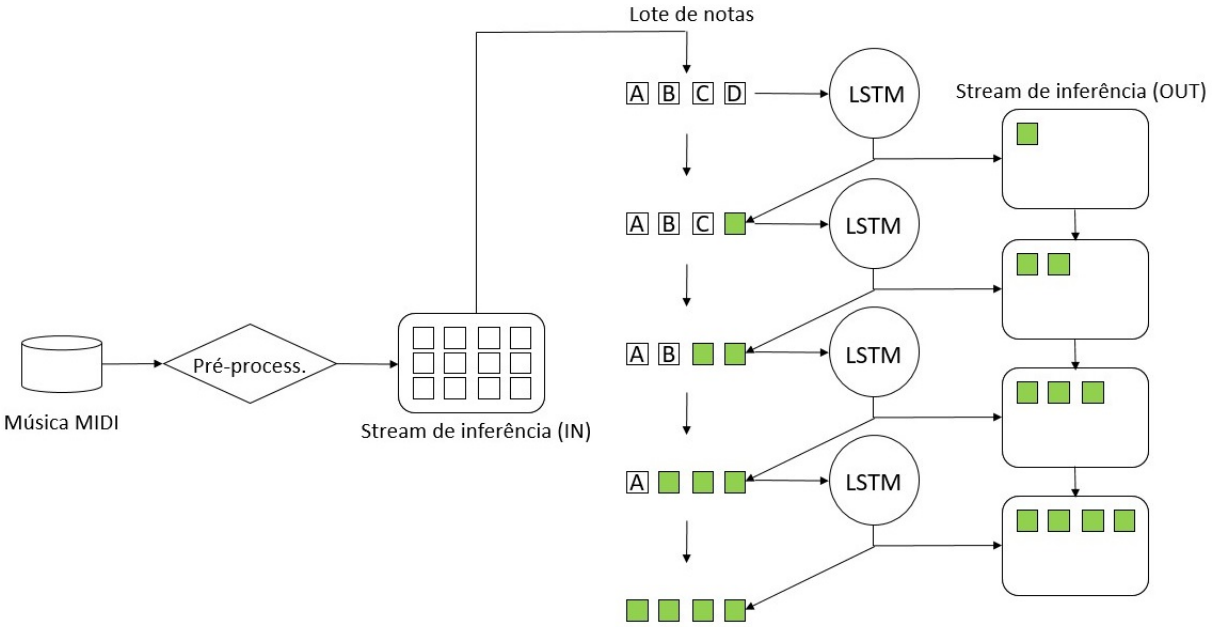
5. Como citado na descrição da modelagem na rede neural, a entrada da mesma recebe um array de tamanho igual ao tamanho do número de itens no dicionário, e o array de entradas. Para adequar o array de entrada atual aos requerimentos da rede neural, usou-se a função 'to_categoricals' da biblioteca Keras, transformando cada array de entrada em dados categóricos do tamanho certo para a rede neural.

6. Após geradas as entradas e saídas, é treinada a rede neural com o uso da função 'fit', pertencente ao modelo 'Model', que recebe as entradas, as saídas, o número de *epochs* e o tamanho do lote, que, como dito anterior mente, é de 64. O modelo então é salvo com o uso da funçãp 'save', que salva as configurações de peso da rede treinada em um arquivo do formato H5.

3.4.2 Inferência de Dados com a LSTM

A parte de geração de música na rede neural é chamada de inferência e é demonstrada na Figura 15, onde os pesos da rede neural já estão ajustados e apresentam alta precisão e onde há um arquivo MIDI que serve de base musical para que se ocorra o início da geração musical, este arquivo então passa pelo algoritmo de pré-processamento e é transformado em uma stream própria (de inferência) para ser processada pela rede neural, é selecionado um lote de eventos contendo N eventos da música base (que agora é uma stream de dados) e então é feita a previsão para o próximo evento do mesmo modo que é feito no treinamento e na validação.

Figura 15 – Inferência Musical com a Rede Neural Treinada.



A diferença é que a predição durante a inferência não é utilizada para ajustar os pesos mas sim para compor a nova música que está sendo gerada, de maneira em que o lote inicial começa a ser substituído pelas predições, que são realimentadas na rede neural e as predições também são todas salvas em uma stream de inferência, que contém as informações sobre cada comando, porém não em formato musical mas sim no formato em que é retirado da rede neural e que deve ser enviado para o pós processamento. O tamanho da música de saída é controlado pelo número de eventos que o usuário que controla o software escolhe.

Na prática, a inferência ocorre da seguinte forma:

1. O usuário fornece quantas notas a rede neural vai inferir;

2. Um array vazio que armazenará os eventos inferidos pela rede neural é criado;

3. Um array *seed* de inteiros com o tamanho do lote a ser processado (64) e contendo valores que variam entre 0 e a quantidade total de itens dentro do dicionário é criado para ser o iniciador da geração musical;

4. A partir daí, começa um ciclo *for* que deverá se repetir pelo número dado no passo 1 ou até quando a rede neural inferir o valor ‘ç’, que, como explicado anteriormente, é o valor que sinaliza o final de uma música, ou seja, caso o valor inferido seja ‘ç’, isso significa que o modelo previu o fim da sua música;

5. O valor seed é modificado e reaproveitado durante cada passo do ciclo *for*, por exemplo: no passo de número 0 (primeiro passo), a *seed* criada no passo 3 é utilizada para a geração de um evento, este evento é então processado (explicação no passo

6) e salvo no array criado no passo 2 e adicionada ao final da *seed*, sendo que o primeiro valor da *seed* é descartado, mantendo-se o tamanho correto da *seed* para ser aproveitada no próximo passo.

6. Como explicado na etapa da modelagem da rede neural, ela não vai retornar apenas um eventon, mas um array com o tamanho do dicionário gerado no pré-processamento, com a distribuição das probabilidades de cada uma das posições do array ser o próximo evento, caso este array seja somado, o valor obtido é 1. O item de maior valor indica que o evento que este item simboliza é o mais provável de ser tocado em seguida, dada a sequência de 64 eventos anteriores que foi processada pela rede neural. Então, para adicionar uma característica de improvisação musical, é criada a funcionalidade de amostragem por temperatura baseada no a função softmax (SHARMA, 2022). Esta funcionalidade recebe um valor que vai de 0 até 1 e, de acordo com este valor, aplica uma variação função softmax na distribuição de probabilidades, quanto mais próximo de 1 o valor da temperatura, mais *hard* a distribuição vai ser, quanto mais próximo de 0, mais *soft* a distribuição será. Por exemplo, dada a distribuição de probabilidades [0.1, 0.5 e 0.2, 1.5, 0.5], uma aplicação da função softmax *hard* faria com que as probabilidades se separassem mais ainda, em algo do tipo [0.02, 0.9, 0.05, 0.028, 0.002], uma aplicação *soft* faria com que as probabilidades se aproximassem, em algo do tipo [0.18, 0.24, 0.22, 0.195, 0.165]. Após a redistribuição das probabilidades com base na temperatura, a função se utiliza de um método da biblioteca Numpy para a seleção semi-aleatória com base no array de probabilidades.

7. Após a conclusão do ciclo *for* citado no passo 4, o array citado no passo 2 está cheio de eventos, em sua forma mais adaptada à rede neural, ou seja, um array cheio de valores inteiros que precisarão ser pós-processados.

3.5 Pós-Processamento

O pós processamento no presente trabalho é a etapa onde se coleta o resultado obtido na saída da LSTM e se converte estas informações para dados úteis, tornando-as em arquivos MIDI novamente, este processo ocorre através de um algoritmo que vai em direção oposta ao algoritmo de pré-processamento. Como explicado na seção anterior, a inferência nos dá um array contedo valores inteiros, que correspondem às posições que os eventos ocupam no dicionário criado durante o pré-processamento, portanto, para transformar este array em um arquivo MIDI, o seguinte é feito:

1. O array contendo os valores inteiros é percorrido valor por valor;

2. Um objeto Music21 do tipo 'Stream', que funciona como um array que armazenará os eventos musicais é criado;

3. Uma variável recebe o valor do tipo string que é a chave na posição que o valor inteiro representa. Esta variável string vai se parecer em algo como '<TIPO>~<VALOR>~<DURAÇÃ

4. A variável é então dividida onde quer que haja o caractere '~' em um array de strings. O item na posição 0 do array vai conter o '<TIPO>', o item na posição 1 vai conter o '<VALOR>' e o item na posição 2 vai conter a '<DURAÇÃO>';

5. Com o auxílio da biblioteca Music21, são criados objetos do tipo 'Note', 'Chord' e 'Rest', com valores e duração variados a depender da informação contida no array de strings gerado no passo anterior. Após a criação destes eventos, eles são adicionados ao objeto criado no passo 2.

6. Após todos os eventos terem sidos adicionados ao objeto citado no passo 2, ele, com o auxílio da funcionalidade *write*, é escrito em um arquivo do tipo MIDI, concluindo assim a geração musical.

4 RESULTADOS

Para a discussão dos resultados, serão demonstrados a aplicação da metodologia discutida no capítulo anterior em três conjuntos, sendo que os dois primeiros são de dados extraídos de trilhas sonoras de jogos diferentes e abrangendo dois consoles diferentes: “Dragon View” (1994) - Super Nintendo Entertainment System (SNES) e “Final Fantasy III” (1990) - Nintendo Entertainment System (NES) como dados de Tipo 1, e um conjunto de 99 músicas aleatoriamente selecionadas dentre todas as músicas dos jogos de NES disponíveis no site www.vgmusic.com com a ajuda de um script de download automatizado escrito na linguagem Python. Para cada uma das bases de dados foram gerados uma stream de dados e um dicionário próprios.

Quatro músicas foram geradas com cada um dos conjuntos de dados, a diferença entre as músicas foi o valor usado na amostragem por temperatura. A primeira música foi gerada com temperatura igual a 1, a segunda com temperatura igual a 0,7, a terceira com temperatura igual a 0,5 e a quarta com temperatura igual a 0,3. Quanto mais próxima de zero a tempratura é, mais “improvisada” a música é, e quanto mais próxima de 1, mais a música segue as indicações de probabilidade geradas pela rede neural.

4.1 Avaliando a Geração Musical com os Dados do Tipo 1

4.1.1 Resultados Obtidos com Músicas do Jogo “Dragon View” - Dados do Tipo 1

O jogo “Dragon View” foi lançado em 1994 e é um RPG que se diferencia dos demais pelo fato de não ser *turn-based*, mas sim um *side-scroller beat ´em up*.

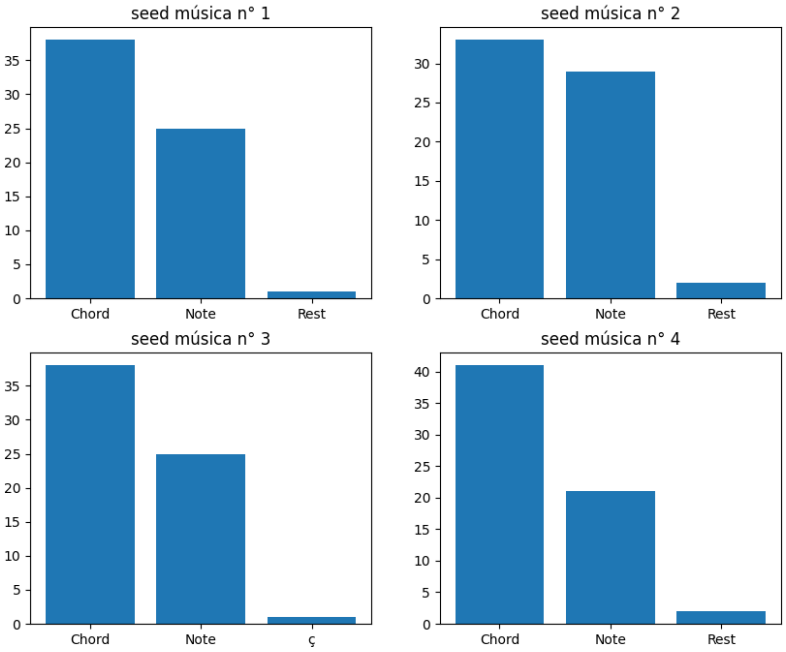
Para a geração musical com este jogo, as seguintes músicas que compunham a trilha sonora foram utilizadas (títulos como descritos em www.vgmusic.com): “Argos Plot”, “Casdra Field”, “Cave of Fire” “Desert Field”, “Distraught”, “Field Battle”, “Fire Field”, “Game Over”, “Mysterious Cave”, “Priest of Keire”, “Ruins of Neil Village”, “Rysis Field”, “Temple of The Lake”, “Strange Illness”, “Swamp Field”, “Title Screen”, “Town of Miraj”, “Town of Rysis” e “Underworld”. Após a transformação destas músicas pelo algoritmo de pré-processamento, obtemos uma stream de dados com 19465 eventos e um dicionário com 108 eventos únicos registrados. A rede neural contém 401516 nós treináveis que se adaptaram para os dados de treinamento com acurácia de 99,14% e se adaptaram com acurácia de 88,98% ao conjunto de validação (80 *epochs*). Isso significa que a rede neural se mostra *underfit*, ou seja, a rede neural não se desempenhou da forma ideal desejada e não pôde generalizar bem a geração musical.

Figura 16 – Capa do Jogo "Dragon View".[1]



As *seeds* utilizadas foram 64 números randômicos contidos nos valores armazenados dentro do dicionário. Os eventos contidos nas seeds eram do tipo:

Figura 17 – Eventos nas *seeds* das quatro músicas - "Dragon View".



[1] Geekquest. Disponível em: <https://geekquest.org/2020/01/28/review-de-dragon-view-nada-melhor-que-uma-boa-aventura-medieval/>, Acesso em nov. 2022

Figura 18 – Quantidades de eventos por tipo nas quatro músicas - “Dragon View”.

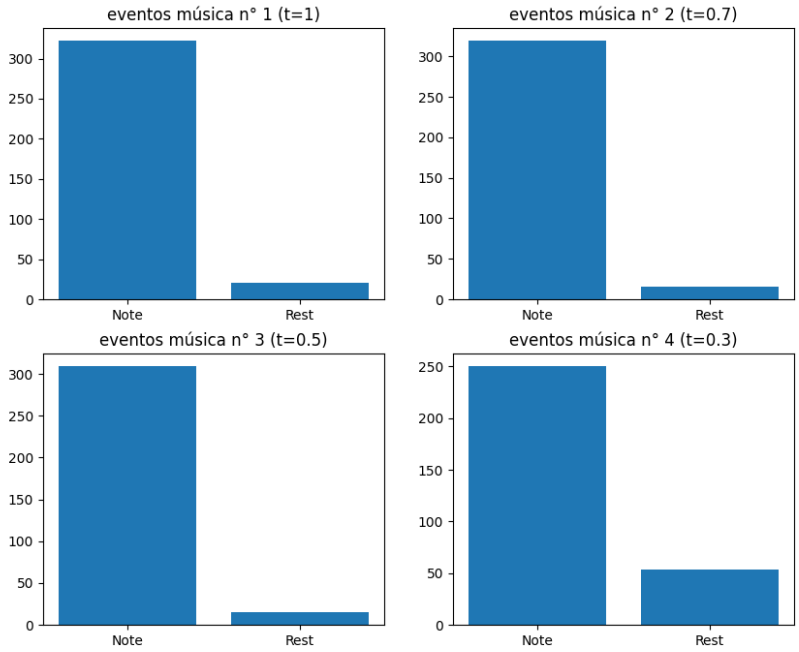


Figura 19 – Quantidade de eventos por tom nas músicas - “Dragon View”.

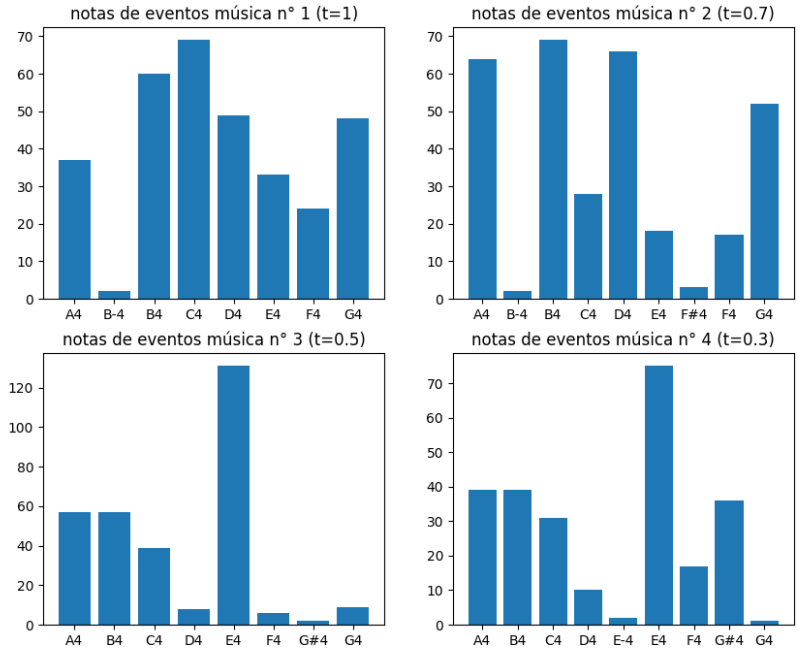
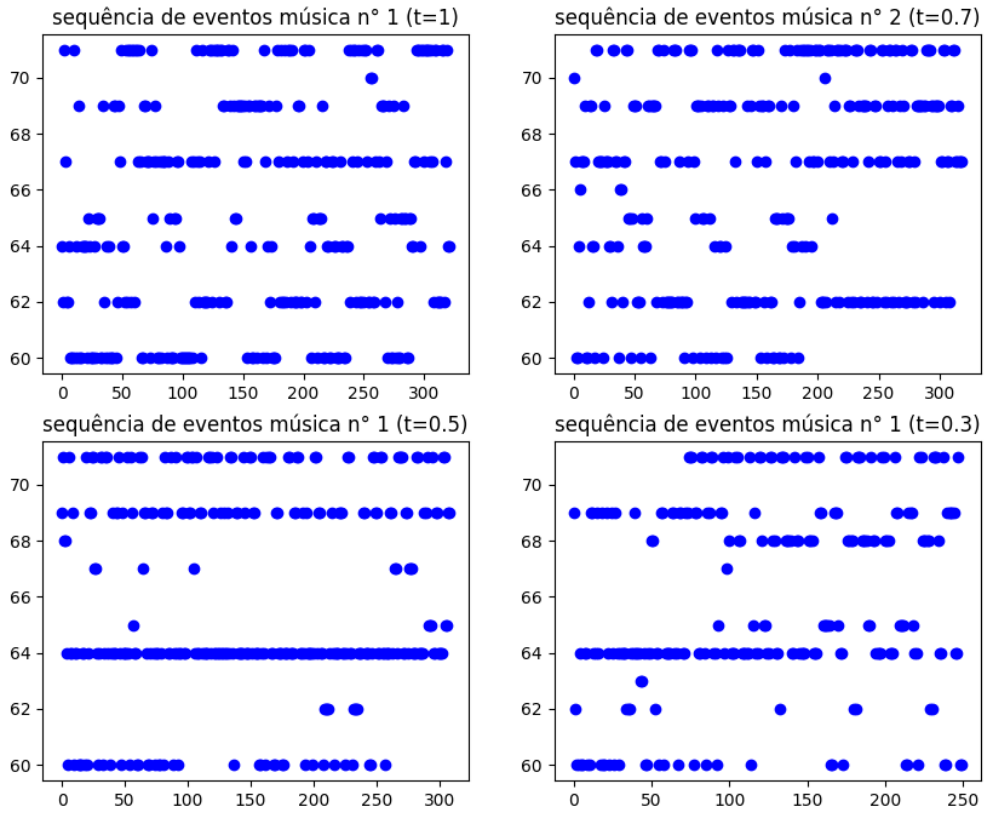


Figura 20 – Sequência de eventos nas quatro músicas - “Dragon View”.



Notas em azul, acordes em vermelho.
Cada número indicado no eixo y é um valor de tom diferente, onde 60 = C4, 61 = C#4, 62 = D4, 63 = D#4, 64 = E4, 65 = F4, 66 = F#4, 67 = G4, 68 = G#4, 69 = A4, 70 = A#4, 71 = B4.

4.1.2 Resultados Obtidos com Músicas do Jogo “Final Fantasy III” - Dados do Tipo 1

O jogo “Final Fantasy III” foi lanado para o sistema NES no japão em 1990 e vendeu 1,4 milhão de cópias lá. É um jogo japonês desenvolvido e publicado pela Square e é do gênero RPG.

Figura 21 – Capa do Jogo “Final Fantasy III”.[2]



Para a geração musical com este jogo, as seguintes músicas que compunham a trilha sonora foram utilizadas (títulos como descritos em www.vgmusic.com): “Airship”, “Boss Battle”, “Castle”, “Crystal Cave”, “Cute Little Tozas”, “Dark World”, “Elia, the Maiden of Water”, “Eternal Wind”, “Everlasting World”, “Forbidden Land”, “Go Above the Clouds!”, “Jinn, the Fire”, “Let Me Know the Truth”, “Lute of Noah”, “Perished”, “Prelude”, “Sylx Tower”, “Time Remains”, “Town of Water”, “Town” e “World Map”. Após a transformação destas músicas pelo algoritmo de pré-processamento, obtemos uma stream de dados com 29787 eventos e um dicionário com 267 eventos únicos registrados. A rede neural contém 605195 nós treináveis que se adaptaram para os dados de treinamento com acurácia de 98,35% e se adaptaram com acurácia de 76,42% ao conjunto de validação (80 *epochs*).

As *seeds* utilizadas foram 64 números randômicos contidos nos valores armazenados dentro do dicionário. Os eventos contidos nas seeds eram do tipo:

[2]Moby Games. Disponível em: <https://www.mobygames.com/game/nes/final-fantasy-iii/cover-art/gameCoverId,159409/>, Acesso em nov. 2022

Figura 22 – Eventos nas *seeds* das quatro músicas - "Final Fantasy III".

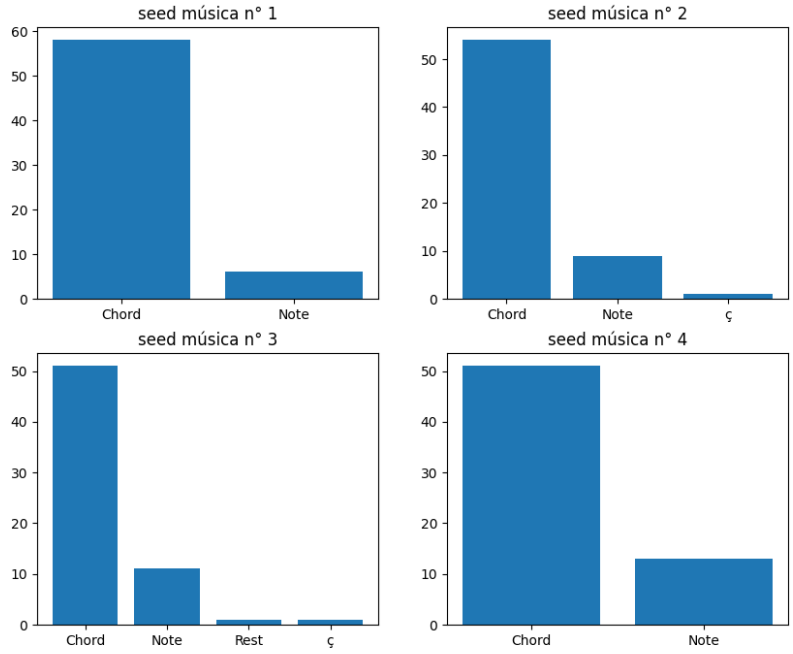


Figura 23 – Quantidades de eventos por tipo nas quatro músicas - "Final Fantasy III".

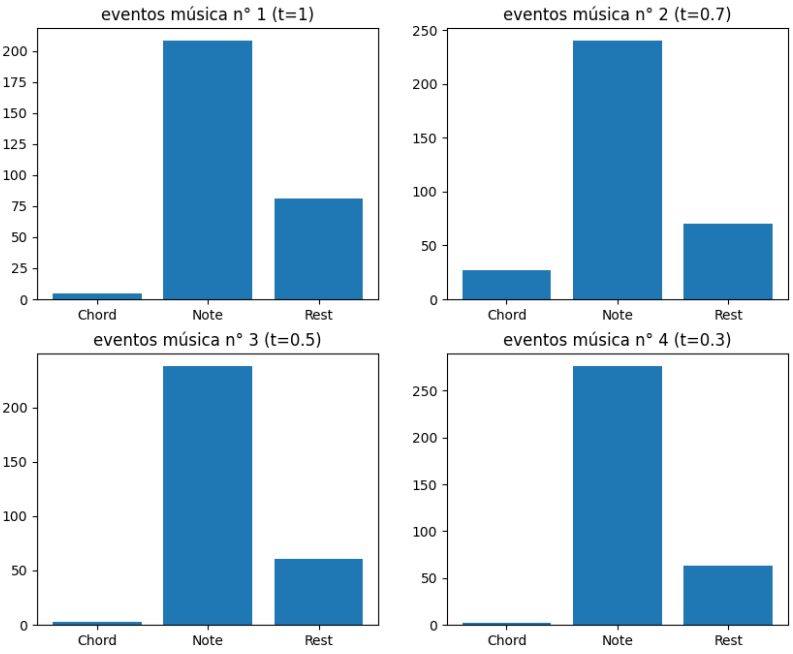


Figura 24 – Quantidade de eventos por tom nas músicas - “Final Fantasy III”.

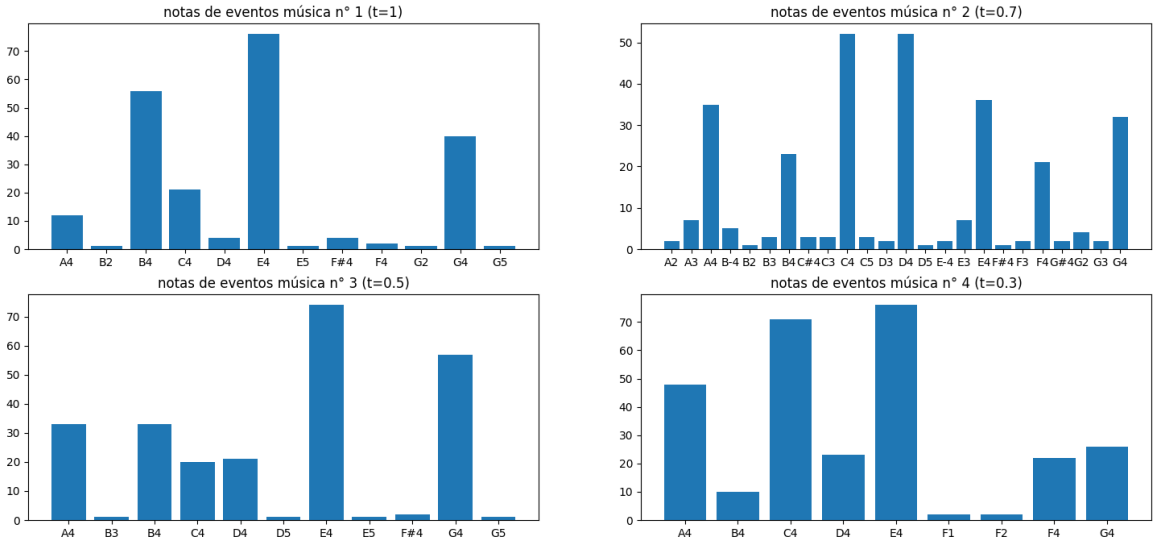
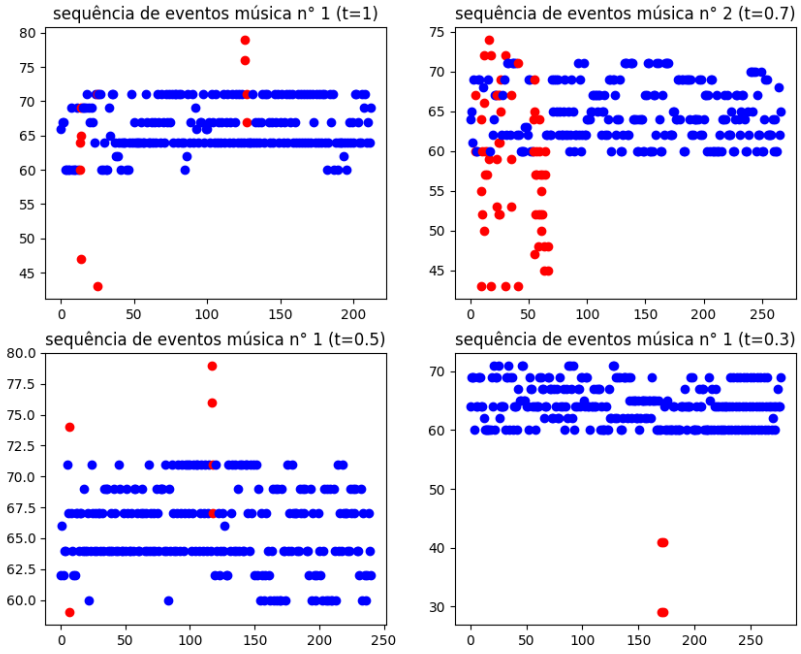


Figura 25 – Sequência de eventos nas quatro músicas - “Final Fantasy III”.



Notas em azul, acordes em vermelho.
Cada número indicado no eixo y é um valor de tom diferente, onde 60 = C4, 61 = C#4, 62 = D4, 63 = D#4, 64 = E4, 65 = F4, 66 = F#4, 67 = G4, 68 = G#4, 69 = A4, 70 = A#4, 71 = B4.

4.1.3 Análise dos Resultados - Dados do Tipo 1

Os resultados obtidos com esses dois conjuntos de dados foram o que deram motivação para a mudança no estilo do pré-processamento, dividindo os dados entre dados

Tipo de 1 e dados Tipo de 2, como explicado na subseção 3.3. Eis aqui perguntas que foram feitas e levadas em consideração para esta mudança:

- Por que a rede neural, quando avaliada com dados de teste se mostraram inferiores em relação ao treinamento?

  Esta discrepância pode ser resumida ao se explicar a particularidade dos acordes e a maneira como estes são codificados para a stream de dados e para o dicionário, pois um acorde pode conter várias notas e diferentes durações, ou seja, se no modelo "<TIPO>~<VALOR>~<DURAÇÃO>", uma simple nota, que contém apenas um <VALOR> pode conter até 8 variações diferentes (variações de duração) e, dentro de uma oitava, como é feito neste trabalho (transposição feita no pré-processamento), todas as notas possíveis gerariam um dicionário de tamanho 96 (12 notas, 8 durações), um acorde, que tem vários valores (notas tocadas em uníssono), pode expandir este dicionário e transformá-lo em um grande peso que acaba por, ao tentar levar em consideração os valores dos acordes e expandir o volume total de dados, atrapalhando mais que ajudando na convergência da rede neural.

- Por que as *seeds* geradas tanto para o jogo 1 quanto para o jogo 2 tinham mais acordes do que notas, sendo que as músicas geradas com os dados do jogo 1 não tinham acordes e as músicas geradas com os dados do jogo 2 tinham tão poucos acordes em relação as notas?

  Novamente, como explicado na pergunta anterior, a caractetística dos acordes permite com que hajam vários acordes de tipos diferentes se repetindo dentro da stream de dados, impedindo assim a convergência da rede neural. Existe mais acordes nas seeds pois existem mais acordes nos dicionários, apesar de seus dados serem de baixo uso estatístico por sempre retornarem com valores mais baixos quando a rede neural analisa um lote de dados. A razão mais plausível para que estes valores não sejam escolhidos nem quando a temperatura é a mais baixa (0.3) se dá porque suas probabilidades de serem escolhidos é tão pequena que, mesmo com a redistribuição da probabilidades, acaba não atingindo um valor próximo outras probabilidades, sendo deixada sempre para trás na escolha semi-randômica.

- Por que, se comparadas, os gráficos da Figuras 19 e 24, que, em teoria, deveriam conter as mesmas colunas, se mostram tão diferentes?

  Na seção do trabalho que trata do pré-processamento de dados, é explicado que as músicas são transpostas para as chaves de C Maior, caso estejam em tom maior e A Menor, caso estejam em tom menor. Esse processo realmente ocorre, as notas correspondentes a estas chaves, na biblioteca Music21, se encotram com os valores entre 60 e 71, como explicado nas Figuras 20 e 25, mas aparentemente as funções que calculam chaves, distâncias e realizam a transposição dos valores dos eventos em

uma música para uma chave diferente somente o fazem com objetos do tipo *Note*, dessa forma, os acordes acabaram por abranger um alcance muito maior de notas, tornando a convergência ainda pior.

Isto também explica a diferença entre as figuras 20 e 25. Os valores do acorde "estouraram" o alcance dos valores das notas, onde os acordes, em vermelho, com os dados do jogo 2, se mostram por todo o lugar enquanto as notas seguem o padrão entre 60 e 71.

Para endossar as ideias geradas a partir da análise dos dados de Tipo 1, foram refatorados os dados utilizados para os questionamentos realizados na nesta subseção em dados do Tipo 2, a fim de que o desempenho da rede neural fosse testado novamente, dessa vez sem o valor dos acordes nos eventos do dicionário e da stream de dados.

Os dados do jogo "Dragon View", após refatoração, geraram 331061 nós treináveis e acurácia de 99,04% para os dados de treino e acurácia de 91,03% para os dados de teste (80 *epochs*), com 19333 eventos na stream de dados e apenas 53 eventos únicos no dicionário. Quando os dados estavam codificados como dados do tipo 1, 19465 eventos foram registrados, sendo que o dicionário continha 108 eventos.

Os dados do jogo "Final Fantasy III", após refatoração, geraram 336185 nós treináveis, acurácia de 99,34% para os dados de treinamento e acurácia de 79,15% para os dados de teste (80 *epochs*), com 29021 eventos na stream de dados e 57 eventos únicos registrados no dicionário. Quando os dados estavam codificados como dados do tipo 1, 29787 eventos foram registrados, sendo que o dicionário continha 267 eventos.

Os dados citados nos dois parágrafos anteriores servem para corroborar com as respostas para as perguntas feitas anteriormente: mesmo com o número de eventos reduzido, os dados de Tipo 2 conseguiram diminuir o tamanho dos dicionários de forma expressiva e, apesar da diminuição de eventos registrados na stream de dados, conseguiram um aumento na acurácia com os dados de teste, ainda que aumentos mínimos (de 88,98% para 91,03% em "Dragon View" e de 76,42% para 79,15% em "Final Fantasy III").

Dadas estas explicações, para onde o trabalho pode ir no sentido de geração musical? O plano inicial de gerar tanto notas quanto acordes acaba tendo que ser modificado, pois os valores dos acordes são muito difíceis de serem manipulados e, quando utilizados do jeito citado anteriormente, criam mais problemas do que soluções, aumentam os dados na stream de dados, porém diminuem a acurácia da mesma e raramente são utilizados por serem tão destoantes uns dos outros. O objetivo então se volta à criação de uma linha de base ao invés de geração de composições musicais mais complexas e específicas, visando criar o modelo ideal para este tipo de rede neural: uma stream de dados com um grande volume de informação, mas com um dicionário menor, onde os eventos da stream de dados incluam o que está contido no dicionário de maneira não-esporádica, criando uma distribuição mais similar de porcentagens. O objetivo de criar músicas usando os

dados específicos de apenas um jogo (para criar uma atmosfera parecida com a do jogo na música criada) se muda em criar uma base de dados grande que inclua músicas de vários jogos e que gere músicas sem uma atmosfera específica.

4.2 Avaliando a Geração Musical com os Dados do Tipo 2

Os dados do Tipo 2 foram retirados de 99 músicas aleatórias extraídas da página de trilhas sonoras dos jogos de NES no site vgmusic (https://www.vgmusic.com/music/console/nintendo/ por um script de download automático.

Quando estes dados são utilizados para treinar a rede neural, as seguintes informações são obtidas: a stream de dados armazena 92474 eventos, o dicionário contém 56 eventos únicos registrados e a rede neural acaba por gerar 336185 parâmetros trináveis. Em relação à acurácia, os dados de treino obtiveram 98,80% e os dados de teste obtiveram 85,34% após 80 *epochs*.

Com aproximadamente 85% de acurácia nos dados de teste, as *seeds* utilizadas foram 64 números randômicos contidos nos valores armazenados dentro do dicionário. Os eventos contidos nas seeds eram do tipo:

Figura 26 – Eventos nas *seeds* das quatro músicas - Dados de Tipo 2.

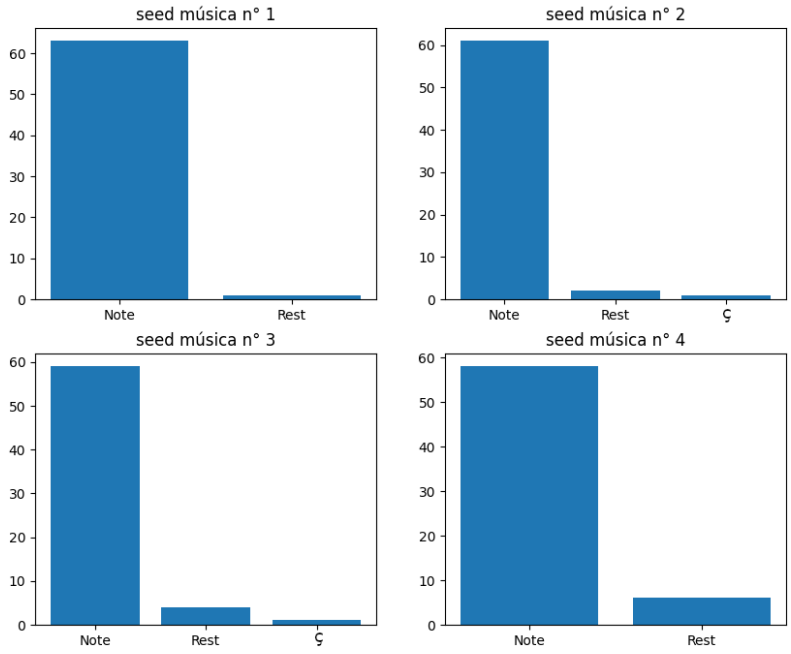


Figura 27 – Quantidades de eventos por tipo nas quatro músicas - Dados de Tipo 2.

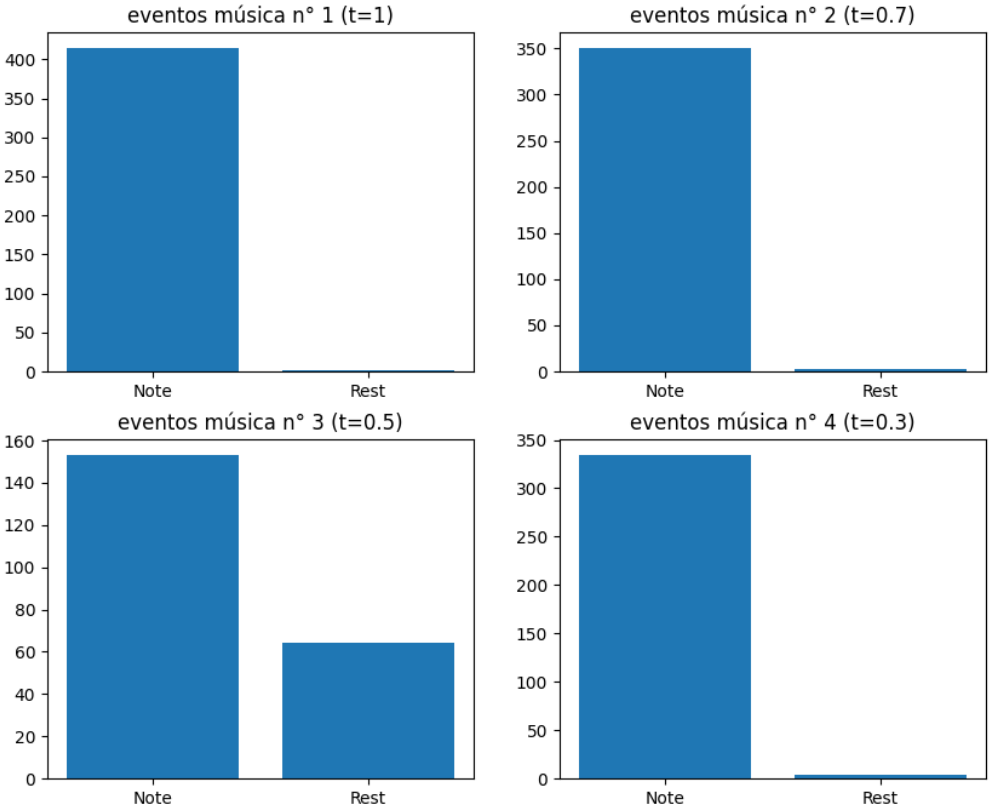


Figura 28 – Quantidade de eventos por tom nas músicas - Dados de Tipo 2.

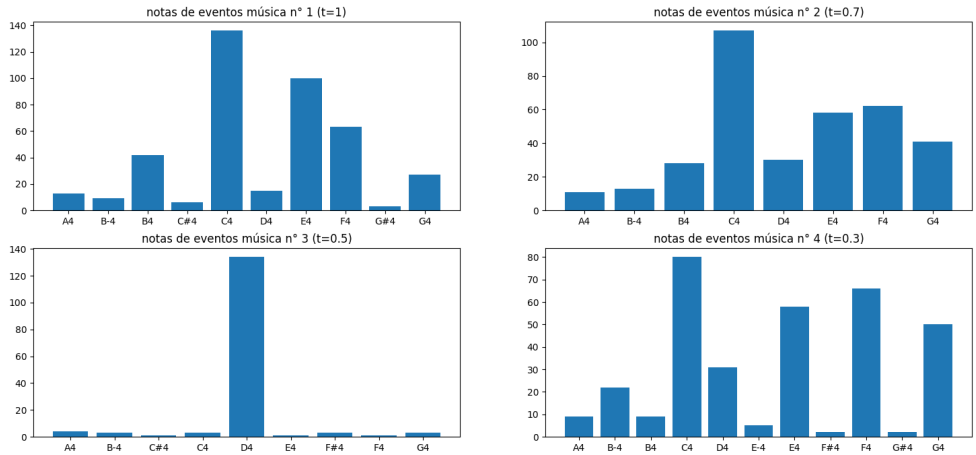
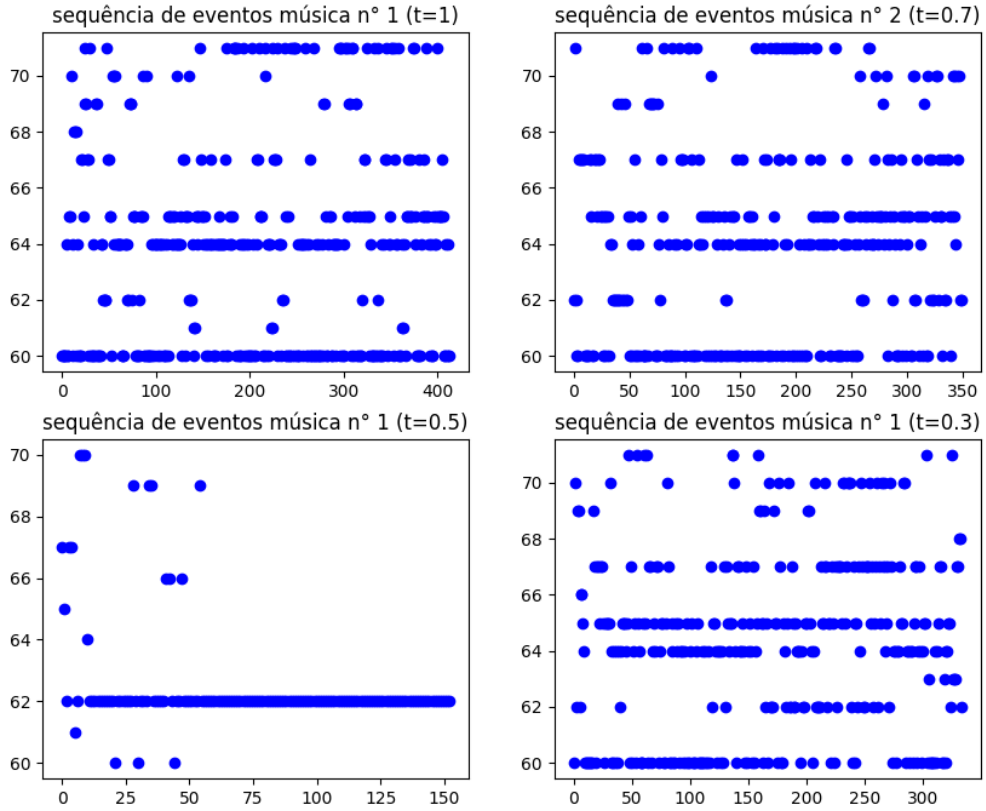


Figura 29 – Sequência de eventos nas quatro músicas - Dados de Tipo 2.



Cada número indicado no eixo y é um valor de tom diferente, onde 60 = C4, 61 = C#4, 62 = D4, 63 = D#4, 64 = E4, 65 = F4, 66 = F#4, 67 = G4, 68 = G#4, 69 = A4, 70 = A#4, 71 = B4.

As quatro músicas MIDI, juntas de suas partituras, podem ser acessadas em shorturl.at/psGPY.

O que pode ser extraído das quatro figuras geradas nesta seção? No geral dá para se dizer que existem três músicas que "funcionaram" e uma música que "não deu certo" (tendo em vista que uma avaliação musical é subjetiva, e o objetivo do trabalho é dar uma base simples para criações musicais). Esta música é a terceira, que foi gerada com temperatura igual a 0,5. O problema com esta música é que ela acaba por cair em um looping de notas a partir de aproximadamente metade da sua duração, contendo somente a nota D4 (Ré, quarta oitava) se repetindo e se intercalando apenas com pausas.

Ainda existe uma comparação a ser feita, que tem a ver com a autenticidade da rede neural: quantos pedaços musicais se repetem em larga escala e em pequena escala. Para avaliar esta informação, foram catalogadas todas as sequências correspondentes a uma barra (quatro tempos musicais) das músicas geradas e comparados com o que está presente na stream de dados para se saber quanta autenticidade há nas músicas.

A música de temperatura igual a 1 continha 297 barras (no total, não em sequência), sendo que dentre estas, 161 foram encontradas dentro da stream de dados, porém isto não é algo necessariamente negativo, pois as músicas geradas pela rede neural foram, de fato, geradas a partir da stream de dados. Isto pode se tornar um problema caso a música gerada seja literalmente igual a alguma música dentro da stream de dados, o que não é o caso, pois quando se avaliam duas barras músicais (8 tempos), a música contém 294 "barras duplas" (no total, não em sequência), porém apenas 31 destas são encontradas de forma literal dentro da rede neural, o que mostra que, apesar de usar os dados contidos na stream de dados para a geração musical, a rede neural também mistura dados de músicas diferentes, criando algo novo através da reutilização de pedaços do que está disponível.

As músicas de temperatura igual a 0,7 e 0,3 obtiveram resultados parecidos com o que foi observado no parágrafo anterior. A música de temperatura 0,7 obteve, com a barra única (quatro tempos), 297 barras, das quais 181 estavam contidas na stream de dados. Com oito tempos, esta música obteve 294 "barras duplas", das quais 43 foram encontradas literalmente na stream de dados. A música de temperatura igual a 0,3 obteve, com a barra única (quatro tempos), 296 barras, das quais, 168 estavam contidas na stream de dados. Com oito tempos, esta música obteve 292 "barras duplas", das quais 17 foram encontradas de forma literal na stream de dados.

A música de temperatura igual a 0,5 serve para mostrar a falha completa na geração musical. Por alguma razão, sua seed randômica possibilita que a rede neural caia em um *loop* que limita profundamente a capacidade de generalização da rede neural. Quando analisada pela métrica da barra única (4 tempos), ela contém 284 barras, das quais 226 são encontradas na stream de dados de forma literal. Quando analisada pela métrica das duas barras (8 tempos), ela contém 271 barras que se são encontradas 201 vezes de forma literal. Este valor das duas barras é muito alto quando comparado ao que foi mostrado nos parágrafos anteriores, o que mostra que a rede neural não pôde criar algo novo. A título de curiosidade, analisando-se por quatro barras (16) tempos, pode se encontrar 258 barras diferentes dentro da música, sendo que 182 destas são encontradas de forma literal dentro da stream de dados.

5 CONCLUSÃO E TRABALHOS FUTUROS

O presente trabalho teve como objetivo criar um sistema que facilitasse a criação musical para músicas de videogame antigos que vem no formato MIDI.

Para tal, foram analisados, testados e aplicados conceitos muito importantes e presentes nos desafios propostos para a área de ciência da computação nos tempos atuais: a manipulação e o uso de dados disponíveis na internet para algo além do óbvio ou do que está visível de na superfície - transformar e retransformar informações.

Desta maneira, foram explicados conceitos de pré-processamento, envolvendo a leitura, codificação e transposição dos dados musicais em arrays de valores inteiros que pudessem ser utilizados para o processamento da rede neural, também foram explicados os conceitos que envolvem as redes neurais em si, desde o *perceptron* até o modelo utilizado para a realização do presente trabalho, a rede neural recorrente do tipo LSTM. Também foram mostrados os desafios ao se manipular dados que, apesar de estarem todos codificados no formato MIDI, que é um "padronizador", variam muito devido a maneira com que são feitos (as músicas utilizadas pelo site vgmusic.com foram enviadas pelos seus usuários), sem contar que músicas de videogame não seguem o padrão da música clássica, de forma a nem sempre possuírem chaves ou estruturas musicais bem definidas.

Para a obtenção dos resultados, foram apresentadas quatro músicas (seção 4.2) processadas por uma rede neural que obteve acurácia de aproximadamente 85%. Os resultados apresentados em três das quatro primeiras músicas geradas se mostram satisfatórios ao que o trabalho se propôs a fazer.

5.1 Trabalhos Futuros

- Teste da rede neural em outros tipos de dados, visto que a biblitoeca Music21 pode suportar dados que não sejam MIDI;
- Dados citados no item anterior podem ser músicas que não sejam de videogame, por exemplo, as músicas clássicas, que mostram uma maior estruturação, o que facilitaria o pré-processamento;
- Tentar balancear, de alguma forma, o uso de acordes para incluí-los na stream de dados;
- Integrar o sistema de reconhecimento de barras repetidas citado na seção 4.2 para apontar automaticamente o nível de autenticidade das músicas geradas.

REFERÊNCIAS

ASSOCIATION, T. M. **MIDI History**. 2014–2021.

BLOCK, H. D. The perceptron: A model for brain functioning. i. **Rev. Mod. Phys.**, American Physical Society, v. 34, p. 123–135, Jan 1962. Disponível em: <https://link.aps.org/doi/10.1103/RevModPhys.34.123>.

CARLING, A. **Introducing Neural Networks**. Sigma Press, 1992. ISBN 9781850581741. Disponível em: <https://books.google.com.br/books?id=ewxpQgAACAAJ>.

CHOI, K.; FAZEKAS, G.; SANDLER, M. **Text-based LSTM networks for Automatic Music Composition**. 2016.

CITRI, A.; MALENKA, R. C. Synaptic plasticity: Multiple forms, functions, and mechanisms. Springer Science and Business Media LLC, v. 33, n. 1, p. 18–41, ago. 2007. Disponível em: <https://doi.org/10.1038/sj.npp.1301559>.

COLLINS, K. From bits to hits: Video games music changes its tune. **Film International**, v. 3, 01 2005.

COLLINS, K. **Game Sound; An Introduction to the History, Theory and Practice of Video Game Music and Sound**. [S.l.: s.n.], 2008. ISBN 9780262270694.

COPE, D.; SILVA, P. David cope and experiments in musical intelligence. In: . [S.l.: s.n.], 2013.

COSTANDI, M. **Neuroplasticity**. MIT Press, 2016. (MIT Press Essential Knowledge series). ISBN 9780262529334. Disponível em: <https://books.google.com.br/books?id=V5veDAAAQBAJ>.

COWAN, J. D. Discussion: McCulloch-pitts and related neural nets from 1943 to 1989. Springer Science and Business Media LLC, v. 52, n. 1-2, p. 73–97, jan. 1990. Disponível em: <https://doi.org/10.1007/bf02459569>.

DAWANI, J. **Hands-On Mathematics for Deep Learning: Build a solid mathematical foundation for training efficient deep neural networks**. Packt Publishing, 2020. ISBN 9781838641849. Disponível em: <https://books.google.com.br/books?id=4ozrDwAAQBAJ>.

DOORNBUSCH, P. Early computer music experiments in australia and england. **Organised Sound**, Cambridge University Press, v. 22, n. 2, p. 297–307, 2017.

DU, K.-L.; SWAMY, M. N. S. Recurrent neural networks. In: . Springer London, 2013. p. 337–353. Disponível em: <https://doi.org/10.1007/978-1-4471-5571-3_11>.

ELMAN, J. L. Finding structure in time. Wiley, v. 14, n. 2, p. 179–211, mar. 1990. Disponível em: <https://doi.org/10.1207/s15516709cog1402_1>.

FREAN, M. A "Thermal"Perceptron Learning Rule. **Neural Computation**, v. 4, n. 6, p. 946–957, 11 1992. ISSN 0899-7667. Disponível em: <https://doi.org/10.1162/neco.1992.4.6.946>.

FRIEDMAN, A. et al. Introduction to neurons. In: _____. [S.l.: s.n.], 2005. v. 1860, p. 1–20. ISBN 978-3-540-23858-4.

GOODFELLOW, I. J.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. Cambridge, MA, USA: MIT Press, 2016. <http://www.deeplearningbook.org>.

GURNEY, K. **An Introduction to Neural Networks**. CRC Press, 2018. Disponível em: <https://doi.org/10.1201/9781315273570>.

HASS, J. **Introduction to Computer Music (Chapter 3)**. 2020.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. **Neural computation**, v. 9, p. 1735–80, 12 1997.

JORDAN, M. I. Serial order: a parallel distributed processing approach. technical report, june 1985-march 1986. 1986. Disponível em: <https://www.osti.gov/biblio/6910294>.

JORDAN, M. I. Chapter 25 - serial order: A parallel distributed processing approach. In: DONAHOE, J. W.; Packard Dorsel, V. (Ed.). **Neural-Network Models of Cognition**. North-Holland, 1997, (Advances in Psychology, v. 121). p. 471–495. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0166411597801112>.

KARLIK, B.; OLGAC, A. V. Performance analysis of various activation functions in generalized mlp architectures of neural networks. **International Journal of Artificial Intelligence and Expert Systems**, Citeseer, v. 1, n. 4, p. 111–122, 2011.

LIN, C. et al. Synaptic strength for convolutional neural network. **arXiv preprint arXiv:1811.02454**, 2018.

LOY, G. Musicians make a standard: The midi phenomenon. **Computer Music Journal**, The MIT Press, v. 9, n. 4, p. 8–26, 1985. ISSN 01489267, 15315169. Disponível em: <http://www.jstor.org/stable/3679619>.

LYU, Q.; WU, Z.; ZHU, J. Polyphonic music modelling with lstm-rtrbm. In: **Proceedings of the 23rd ACM International Conference on Multimedia**. New York, NY, USA: Association for Computing Machinery, 2015. (MM '15), p. 991–994. ISBN 9781450334594. Disponível em: <https://doi.org/10.1145/2733373.2806383>.

MANGAL, S.; MODAK, R.; JOSHI, P. Lstm based music generation system. **arXiv preprint arXiv:1908.01080**, 2019.

MARSLAND, S. **Machine Learning: An Algorithmic Perspective, Second Edition**. CRC Press, 2014. (Chapman & Hall). ISBN 9781466583337. Disponível em: <https://books.google.com.br/books?id=6GvSBQAAQBAJ>.

MAZZOLA, G. et al. A short history of midi. In: _____. **Basic Music Technology: An Introduction**. Cham: Springer International Publishing, 2018. p. 115–116. ISBN 978-3-030-00982-3. Disponível em: <https://doi.org/10.1007/978-3-030-00982-3_11>.

MCCULLOCH, W. S.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. Springer Science and Business Media LLC, v. 5, n. 4, p. 115–133, dez. 1943. Disponível em: <https://doi.org/10.1007/bf02478259>.

MINSKY, M.; PAPERT, S. **Perceptrons: An Introduction to Computational Geometry**. Cambridge, MA, USA: MIT Press, 1969.

NEW NAVY DEVICE LEARNS BY DOING; Psychologist Shows Embryo of Computer Designed to Read and Grow Wiser. **The New York Times**, jul. 1958. Disponível em: <https://www.nytimes.com/1958/07/08/archives/new-navy-device-learns-by-doing-psychologist-shows-embryo-of.html>.

NORIEGA, L. Multilayer perceptron tutorial. **School of Computing. Staffordshire University**, Citeseer, 2005.

OLAZARAN, M. A sociological study of the official history of the perceptrons controversy. SAGE Publications, v. 26, n. 3, p. 611–659, ago. 1996. Disponível em: <https://doi.org/10.1177/030631296026003005>.

PAPADOPOULOS, G.; WIGGINS, G. Ai methods for algorithmic composition: A survey, a critical view and future prospects. In: . [S.l.: s.n.], 1999.

ROCKTäSCHEL, T. et al. **Reasoning about Entailment with Neural Attention**. 2015.

ROSENBLATT, F. The perceptron: a probabilistic model for information storage and organization in the brain. **Psychological review**, American Psychological Association, v. 65, n. 6, p. 386, 1958.

ROSENBLATT, F. **Principles of neurodynamics. perceptrons and the theory of brain mechanisms**. [S.l.], 1961.

SCHMIDT-JONES, C. Understanding basic music theory. Rice University, 2013.

SHARMA, H. **Softmax Temperature**. 2022. <https://medium.com/mlearning-ai/softmax-temperature-5492e4007f71/>. Último acesso em 1/11/2022.

SHARMA, S.; SHARMA, S.; ATHAIYA, A. Activation functions in neural networks. **towards data science**, v. 6, n. 12, p. 310–316, 2017.

SMAGT, P. van der; KROSE, B. An introduction to networks neural. 09 1995.

SMAGULOVA, K.; JAMES, A. P. Overview of long short-term memory neural networks. In: **Modeling and Optimization in Science and Technologies**. Springer International Publishing, 2019. p. 139–153. Disponível em: <https://doi.org/10.1007/978-3-030-14524-8_11>.

SQUIRE, L. et al. **Fundamental Neuroscience**. Elsevier Science, 2008. ISBN 9780080561028. Disponível em: <https://books.google.com.br/books?id=GOxrtYzmixcC>.

TAUD, H.; MAS, J. Multilayer perceptron (MLP). In: **Geomatic Approaches for Modeling Land Change Scenarios**. Springer International Publishing, 2017. p. 451–455. Disponível em: <https://doi.org/10.1007/978-3-319-60801-3_27>.

UITDENBOGERD, A. L.; ZOBEL, J. Manipulation of music for melody matching. In: **Proceedings of the Sixth ACM International Conference on Multimedia**. New York, NY, USA: Association for Computing Machinery, 1998. (MULTIMEDIA '98), p. 235–240. ISBN 0201309904. Disponível em: <https://doi.org/10.1145/290747.290776>.

WANG, S.; JIANG, J. **Learning Natural Language Inference with LSTM**. 2015.

ZHAO, K. et al. An emotional symbolic music generation system based on lstm networks. In: **2019 IEEE 3rd Information Technology, Networking, Electronic and Automation Control Conference (ITNEC)**. [S.l.: s.n.], 2019. p. 2039–2043.